

二叉树

- 树的概念
- 树的表示
- 树的遍历

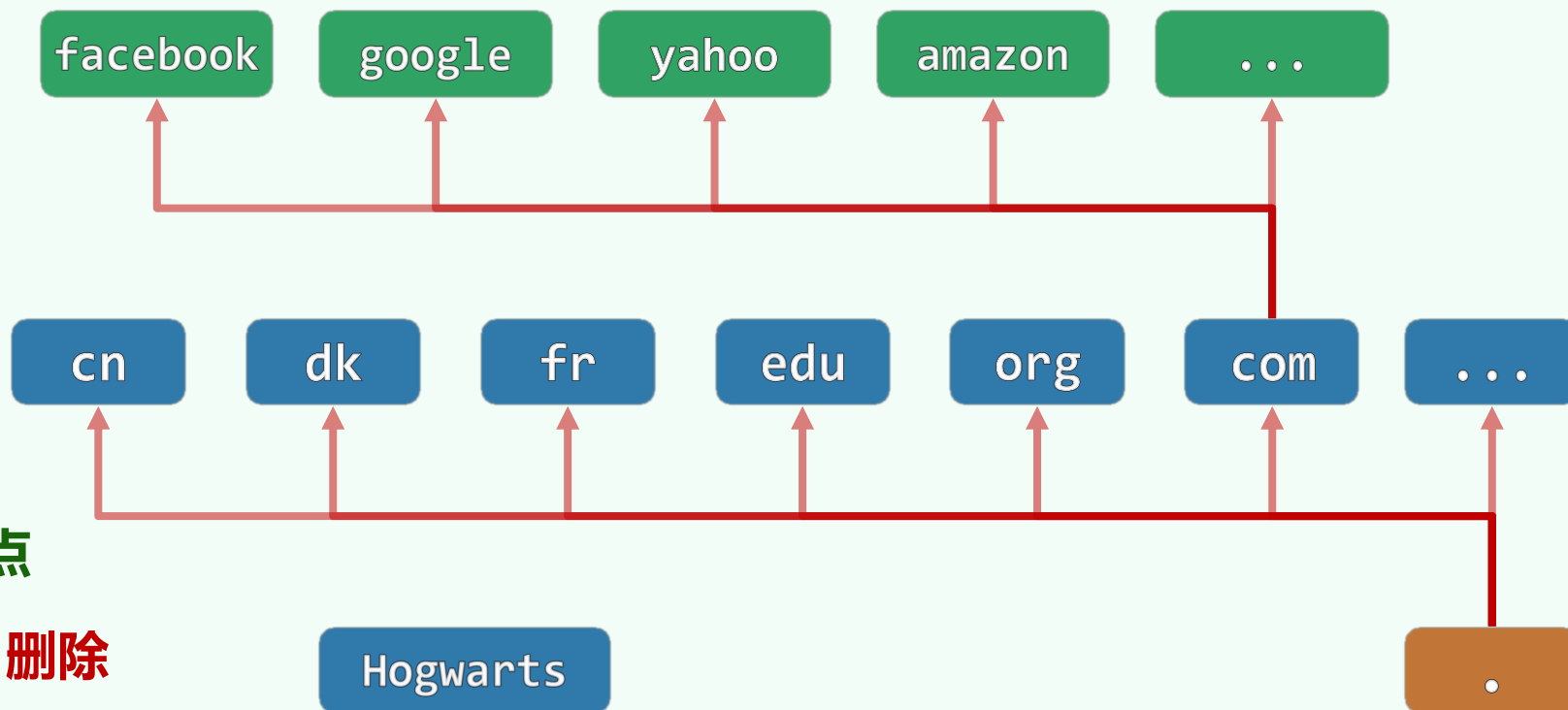
W

二叉树

动机

❖ 【应用】层次结构的表示

- 表达式
- 文件系统
- URL ...

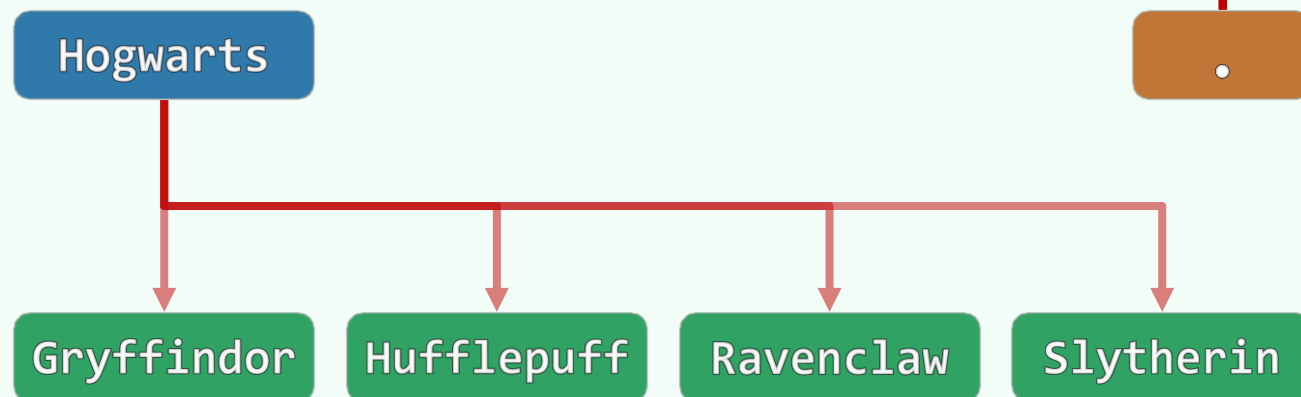


❖ 【数据结构】综合性

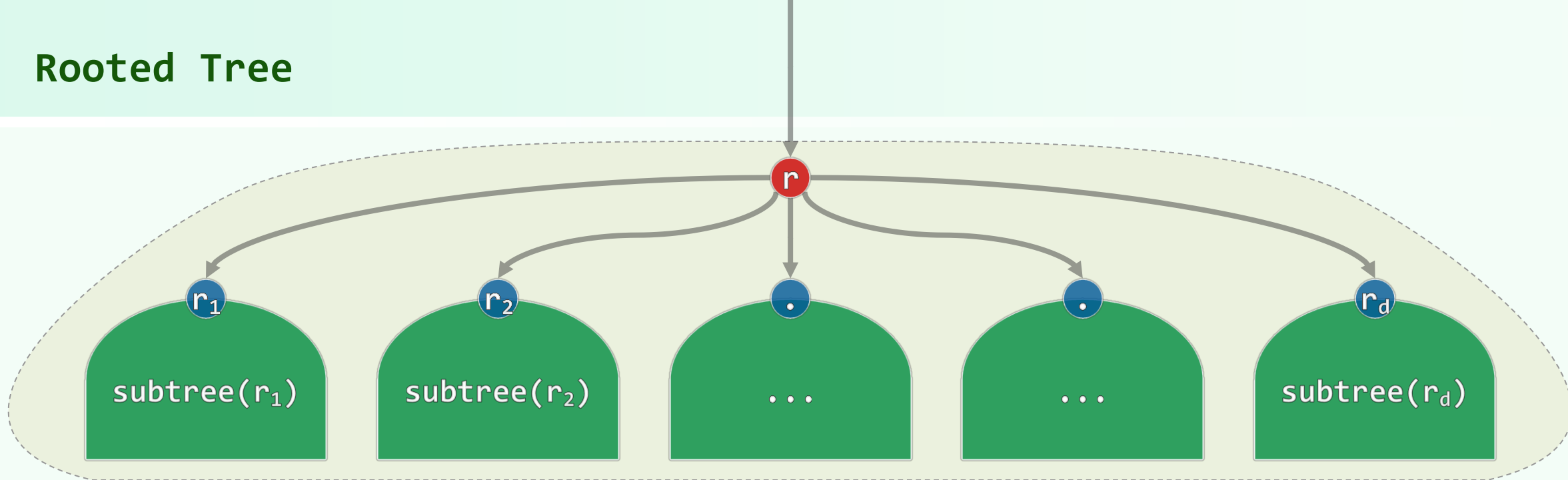
- 兼具Vector和List的优点
- 兼顾高效的**查找**、**插入**、**删除**

❖ 【半线性】

- 不再是简单的线性结构，但
- 在确定某种**次序**之后，具有线性特征

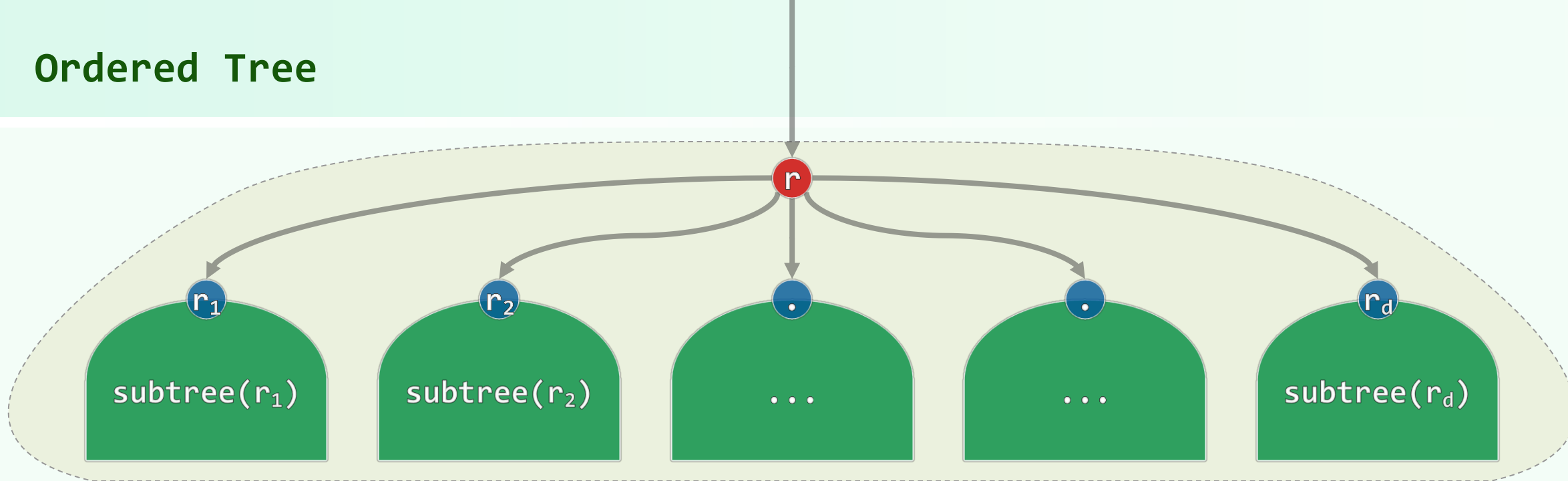


Rooted Tree



- ❖ 树是极小连通图、极大无环图 $\mathcal{T} = (V; E)$: 节点数 $n = |V|$, 边数 $e = |E|$
- ❖ 指定任一节点 $r \in V$ 作为根后, $\mathcal{T}$ 即称作有根树
- ❖ 若 $\mathcal{T}_1, \mathcal{T}_2, \mathcal{T}_3, \dots, \mathcal{T}_d$ 为有根树, 则 $\mathcal{T} = ((\bigcup_i V_i) \cup \{r\}, (\bigcup_i E_i) \cup \{ \langle r, r_i \rangle \mid 1 \leq i \leq d \})$ 也是
- ❖ 相对于 $\mathcal{T}$, $\mathcal{T}_i$ 称作以 r_i 为根的子树 (subtree rooted at r_i), 记作 $\mathcal{T}_i = subtree(r_i)$

Ordered Tree



❖ r_i 称作 r 的**孩子** (child) , r_i 之间互称**兄弟** (sibling)

r 为其**父亲** (parent) , $d = \text{degree}(r)$ 为 r 的 (出) **度** (degree)

❖ 可归纳证明: $e = \sum_{v \in V} \text{degree}(v) = n - 1 = \Theta(n)$

故在衡量相关复杂度时, 可以 n 作为参照

❖ 若指定 $\mathcal{T}_i$ 作为 $\mathcal{T}$ 的第 i 棵子树, r_i 作为 r 的第 i 个孩子, 则 $\mathcal{T}$ 称作**有序树**

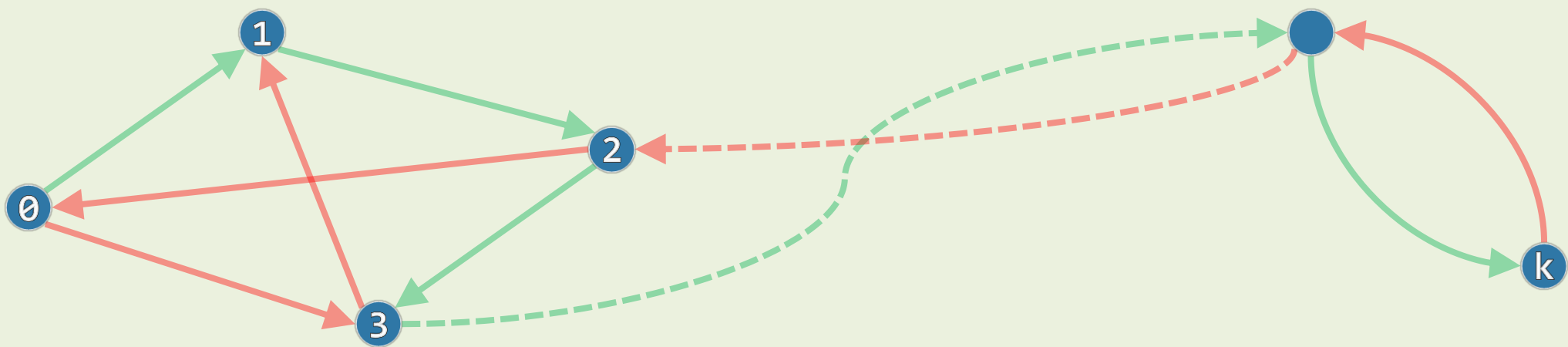
路径 + 环路

❖ V 中的 $k + 1$ 个节点, 通过 V 中的 k 条边依次相联, 构成一条**路径** (path) //亦称通路

$$\pi = \{ (v_0, v_1), (v_1, v_2), (v_2, v_3), \dots, (v_{k-1}, v_k) \}$$

❖ **路径长度**即所含**边数**: $|\pi| = k$ //注意: 早期文献, 多以**节点数**为长度

❖ **环路** (cycle/loop) : $v_k = v_0$ //如果覆盖所有节点各一次, 则称作**周游** (tour)



连通 + 无环

❖ **连通图**：节点之间均有路径 (connected)

不含环路，称作**无环图** (acyclic)

❖ 树：**无环连通图**

极小连通图

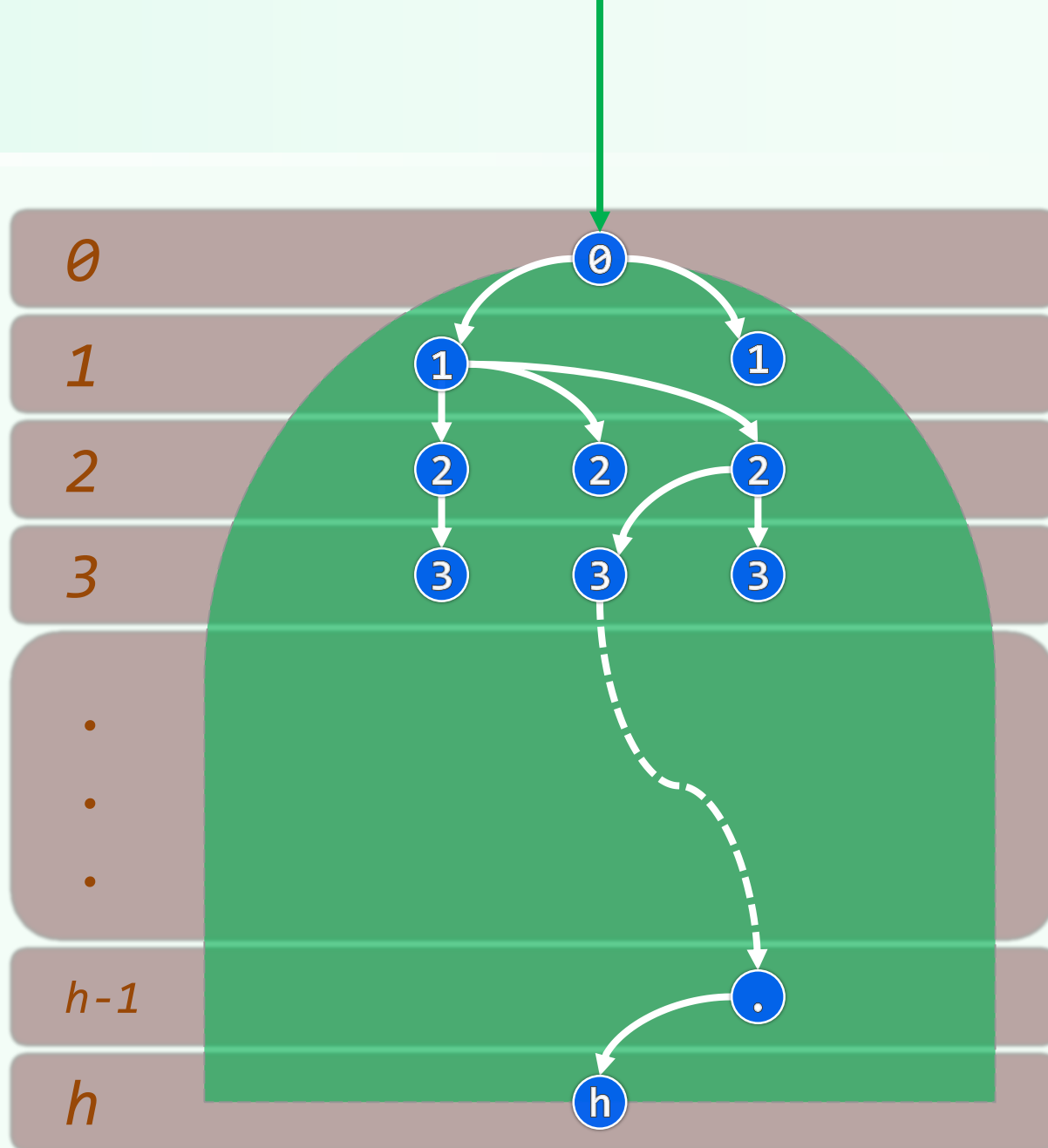
极大无环图

❖ 故：任一节点 v 与根之间存在**唯一**路径

$$\text{path}(v, r) = \text{path}(v)$$

❖ 于是：以 $|\text{path}(v)|$ 为指标

可对所有节点做**等价类**划分...



深度 + 层次

❖ 不致歧义时，路径、节点和子树可**相互指代**

- $\text{path}(v) \sim v \sim \text{subtree}(v)$

❖ v 的**深度**: $\text{depth}(v) = |\text{path}(v)|$

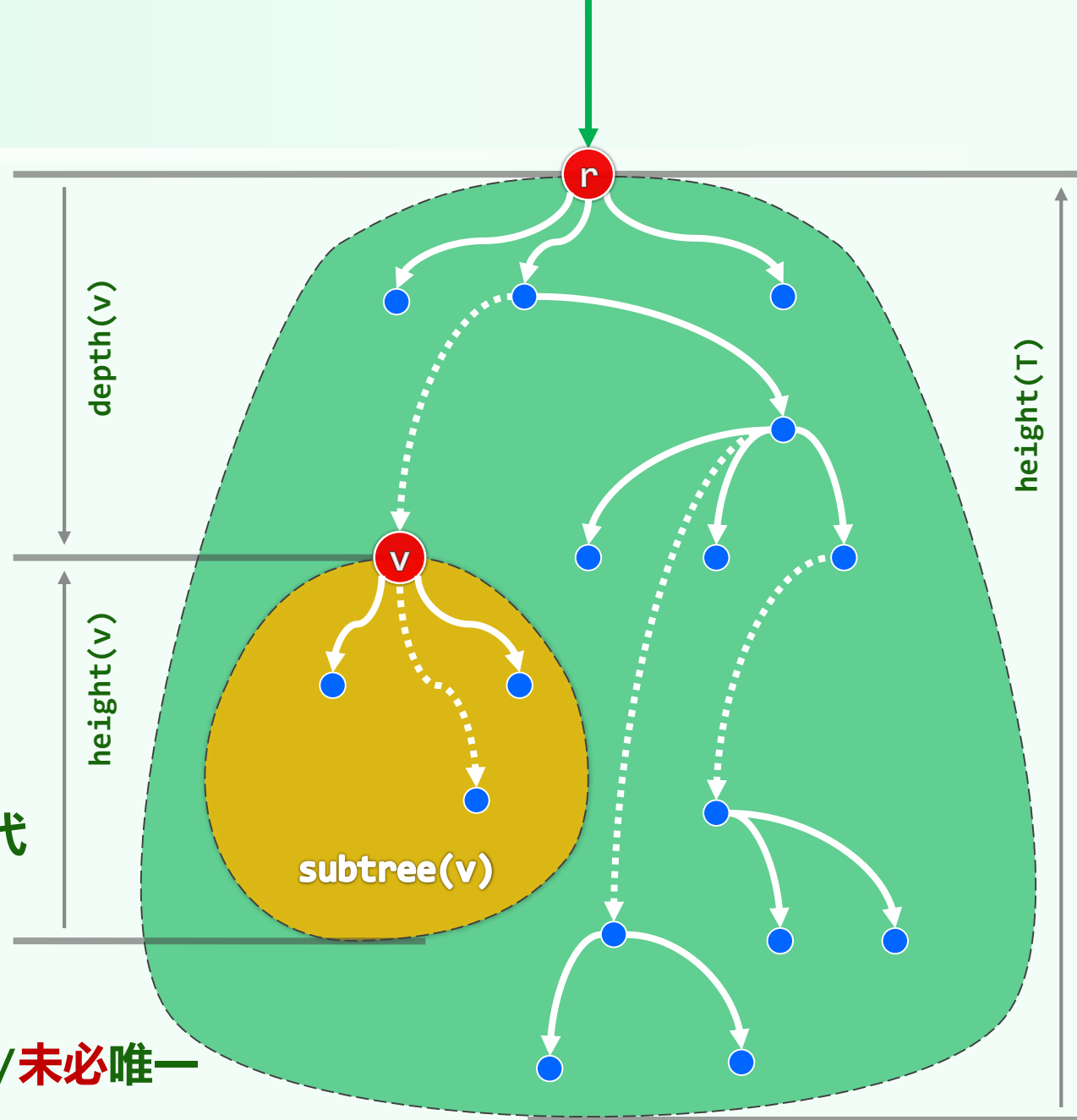
❖ $\text{path}(v)$ 上节点，均为 v 的**祖先** (ancestor)

v 是它们的**后代** (descendent)

❖ 其中除自身以外，是**真** (proper) 祖先/后代

❖ **半线性**:

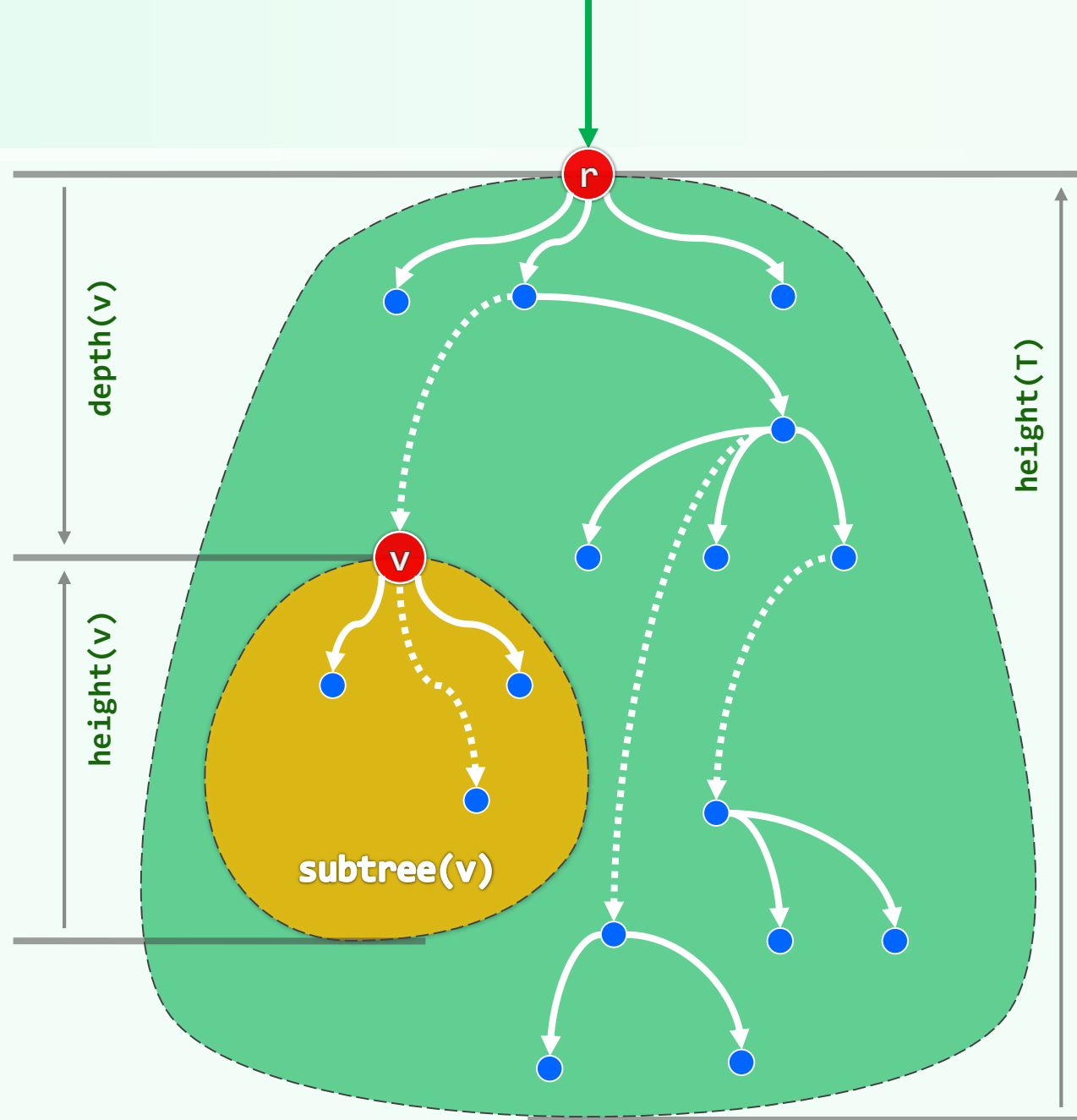
在任一深度， v 的祖先/后代若存在，则**必然/未必唯一**



深度 + 层次

- ❖ 根节点是所有节点的**公共祖先**，深度为0
- ❖ 没有后代的节点称作**叶子** (leaf)
- ❖ 所有叶子深度中的最大者
称作 (子) 树 (根) 的**高度**
 - $\text{height}(v) = \text{height}(\text{subtree}(v))$
- ❖ 特别地，**空树**的高度取作-1
- ❖ $\text{depth}(v) + \text{height}(v) \leq \text{height}(T)$

何时取等号？



二叉树 树的表示

父节点

rank	data	parent
0	H	2
1	E	7
2	F	9
3	B	4
4	R	4
5	K	2
6	D	7
7	A	4
8	G	2
9	C	4

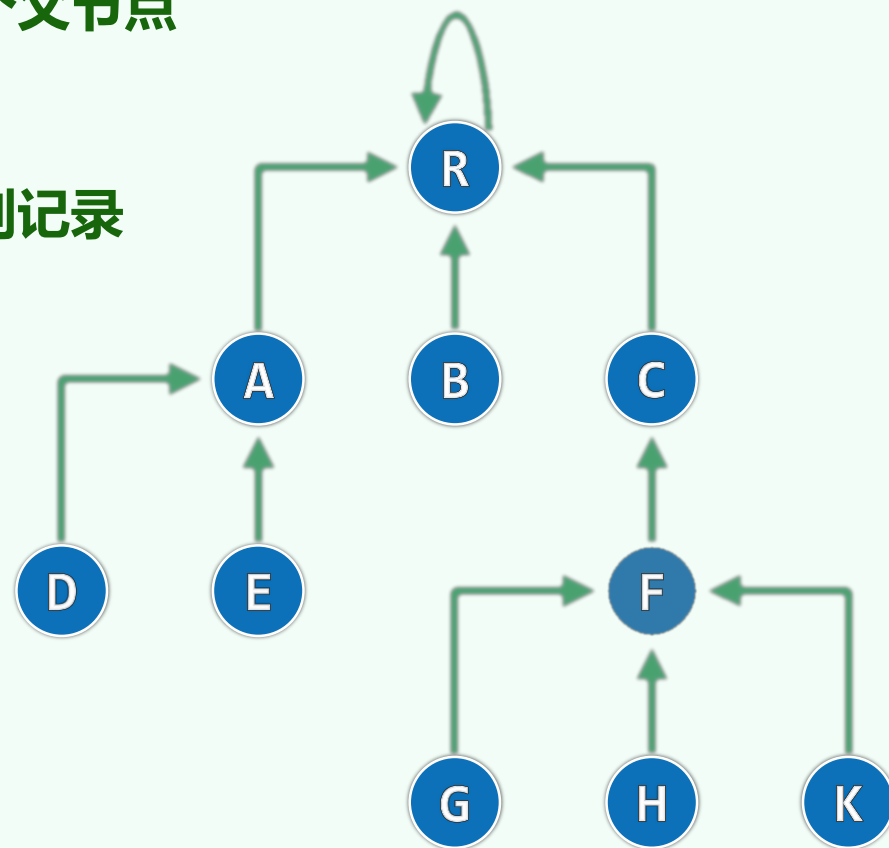
❖ 观察：除根外，任一节点**有且仅有一个**父节点

❖ 构思：将节点组织为序列，各节点分别记录

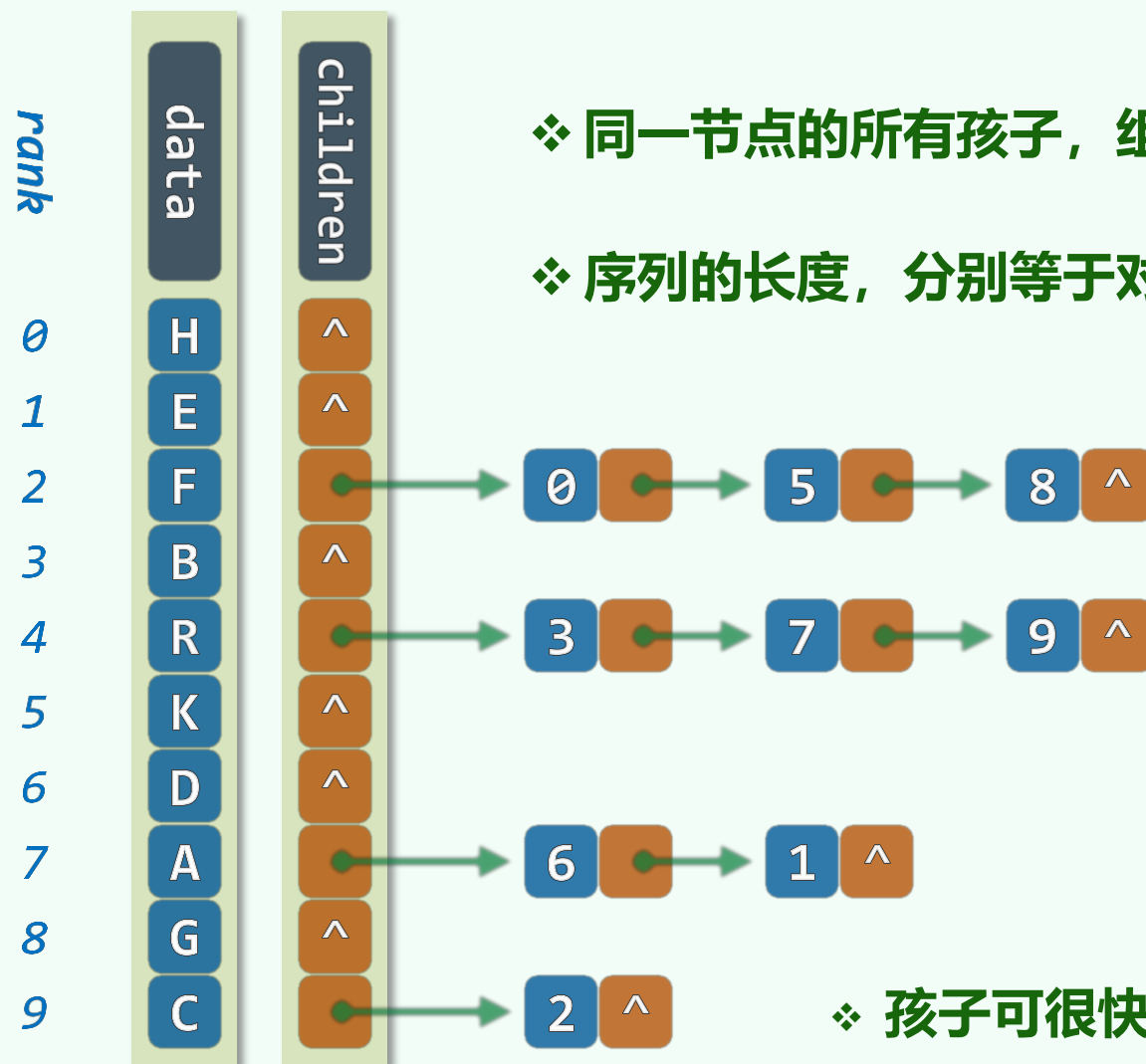
- data 本身信息

- parent 父节点的秩或位置

❖ 树根：R ~ $\text{parent}(4) = 4$

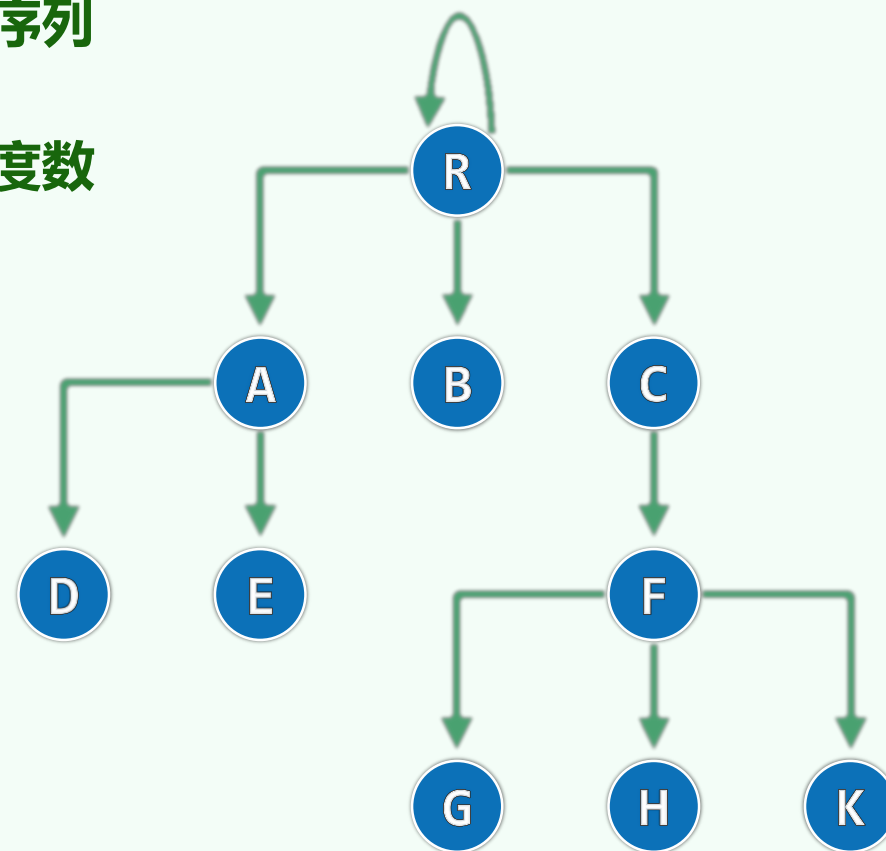


孩子节点



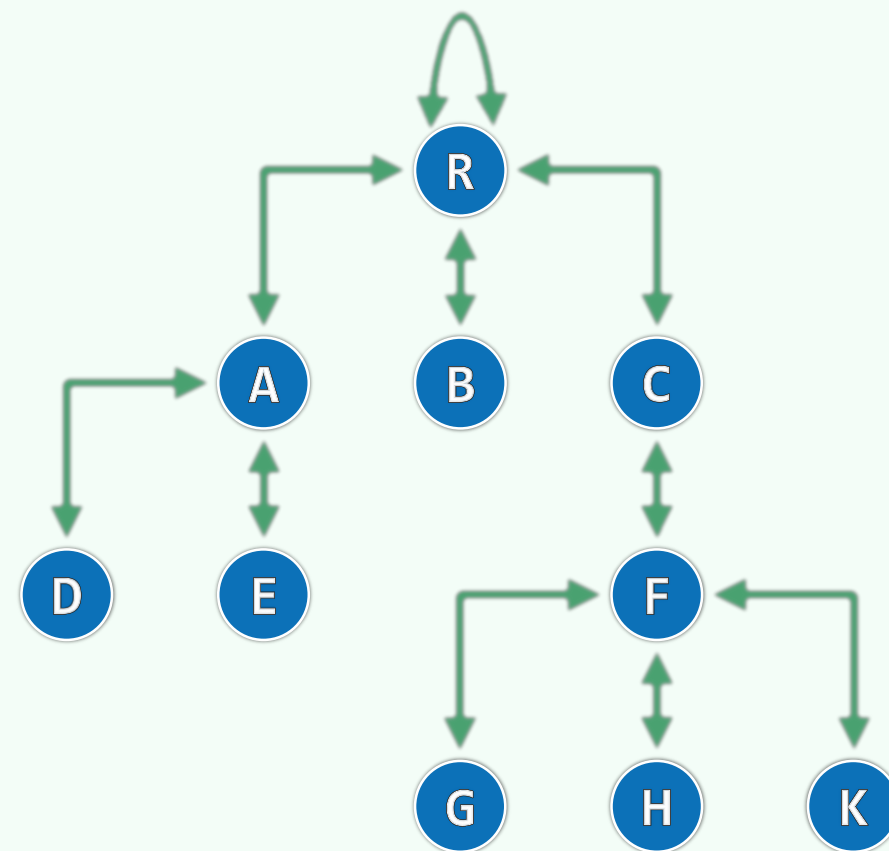
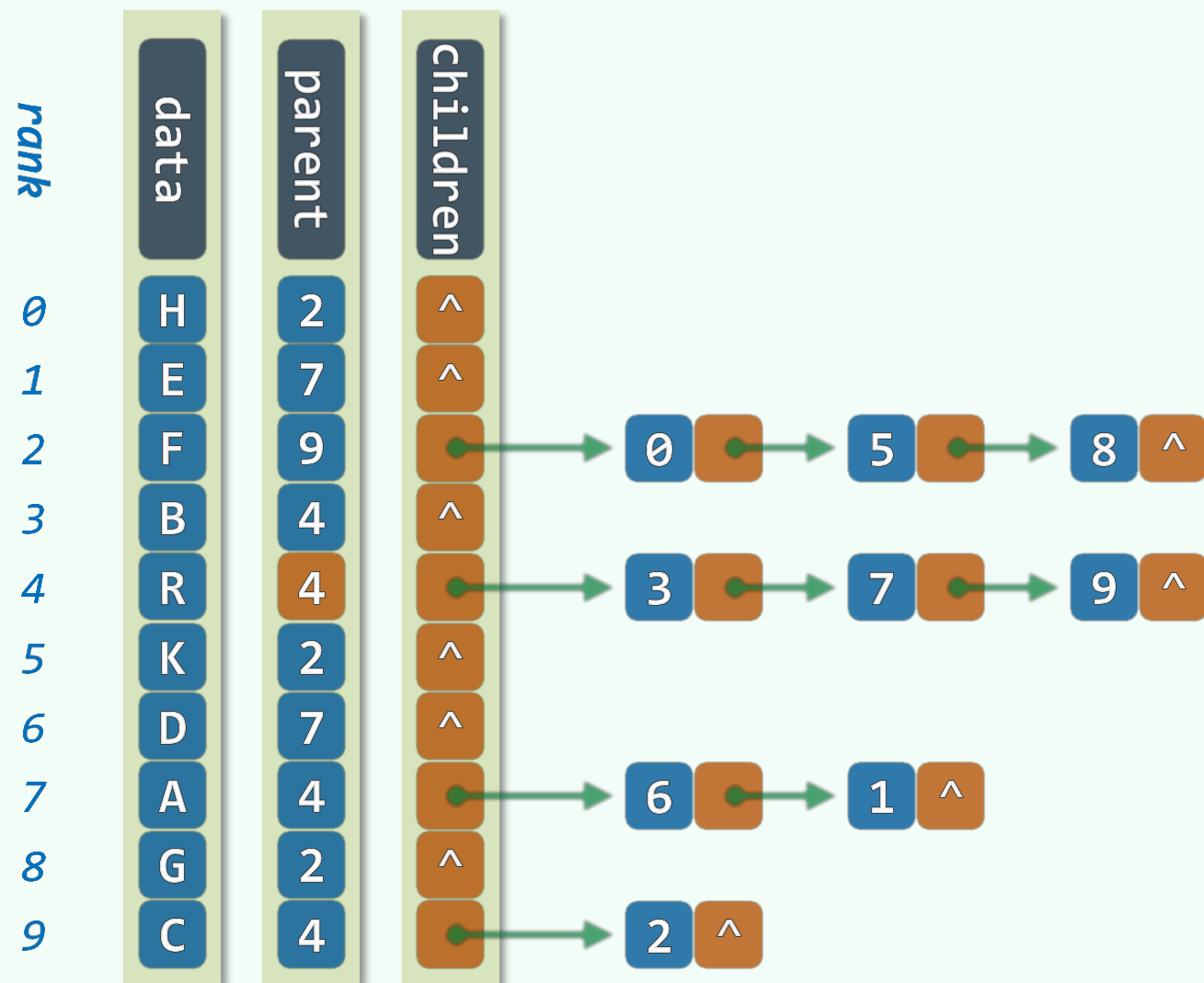
❖ 同一节点的所有孩子，组织为一个序列

❖ 序列的长度，分别等于对应节点的度数



❖ 孩子可很快找出，但parent()却很慢...

父节点 + 孩子节点

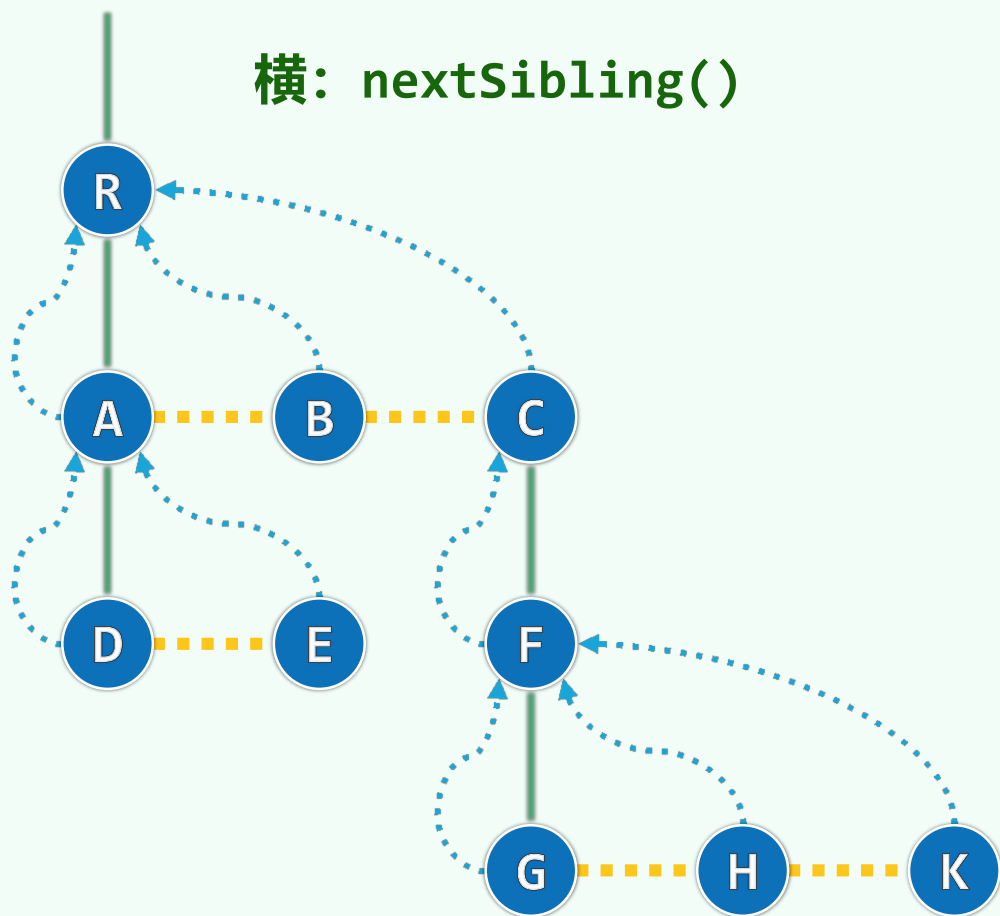


长子 + 兄弟

❖ 每个节点均设两个引用

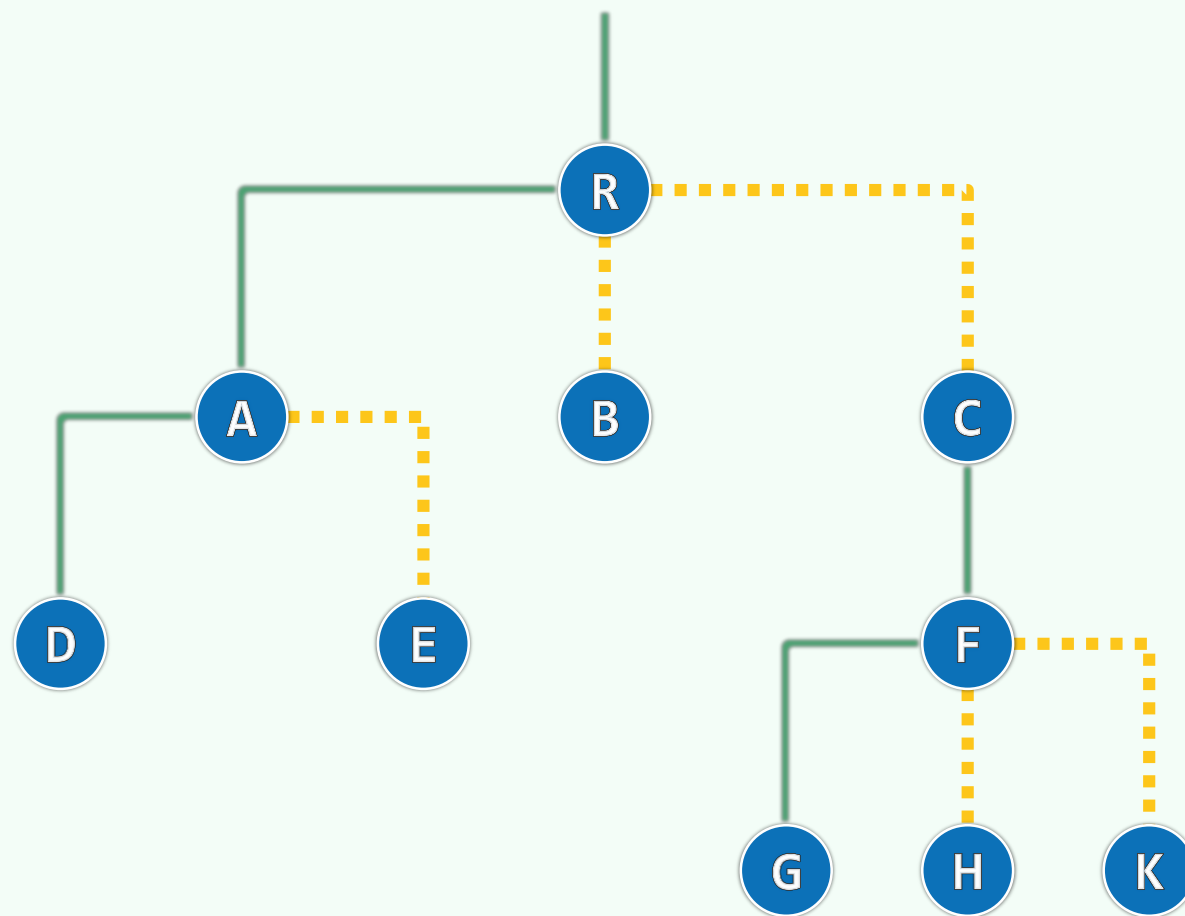
纵: `firstChild()`

横: `nextSibling()`



❖ 用 $O(\deg(v) + 1)$ 时间, 可遍历 v 的所有孩子

❖ 再引入 `parent`, 便可在 $O(1)$ 时间内确定父亲



二叉树

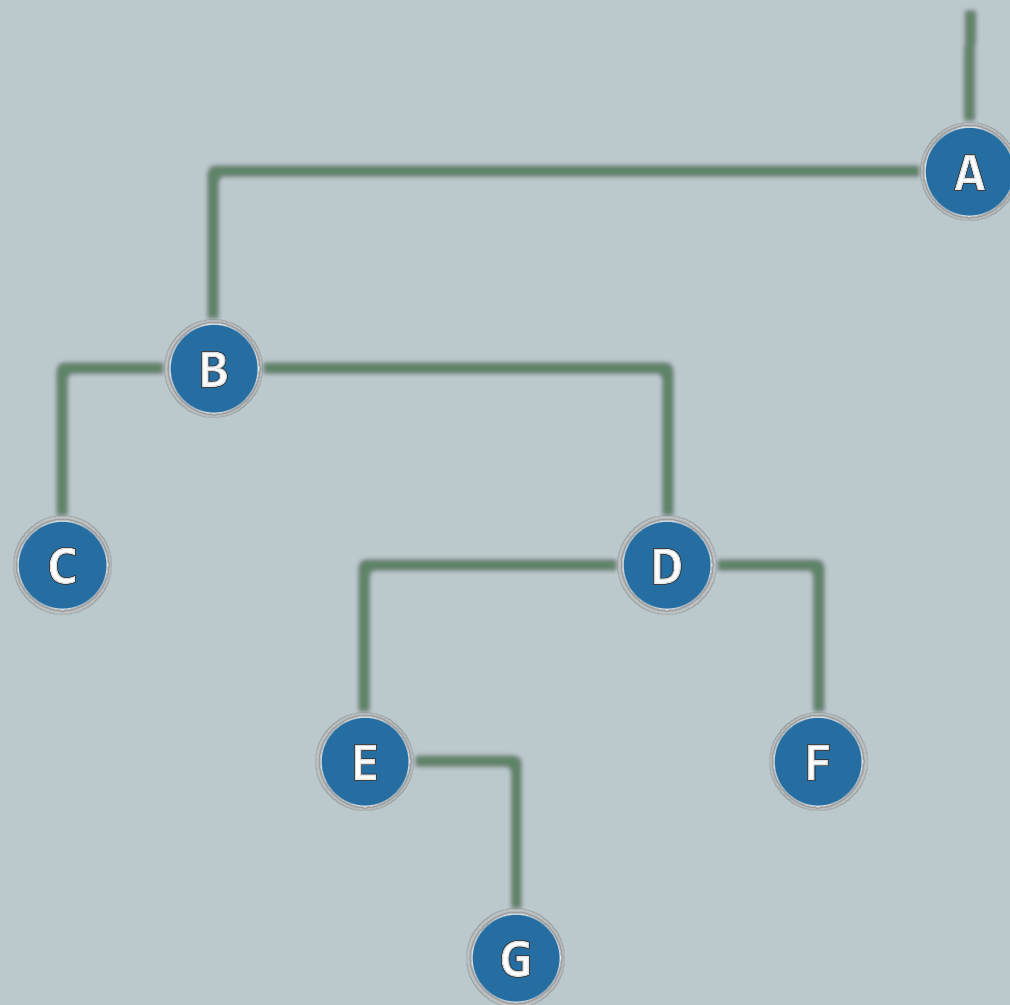
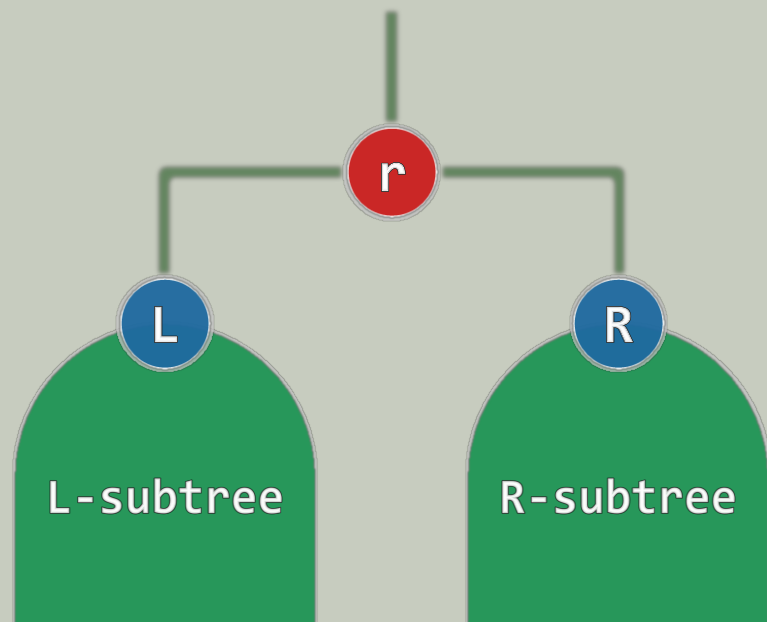
有根有序树 = 二叉树

二叉树

❖ Binary Tree: 节点度数不超过2

孩子（子树）可以左、右区分（隐含有序）

- lc() ~ lSubtree()
- rc() ~ rSubtree()

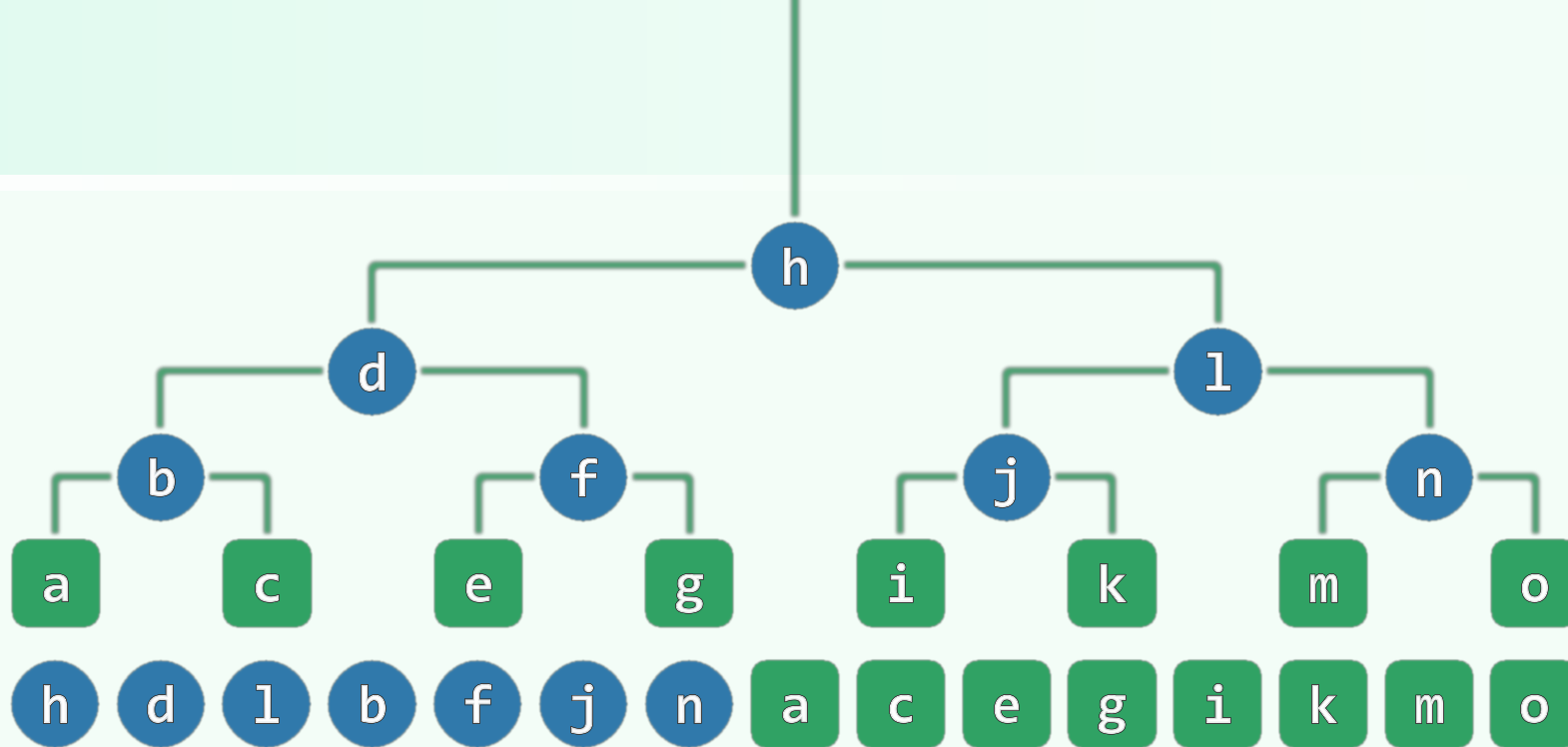


基数 ~ 满树

❖ 深度为k的节点，至多 2^k 个

❖ n个节点、高h的二叉树满足

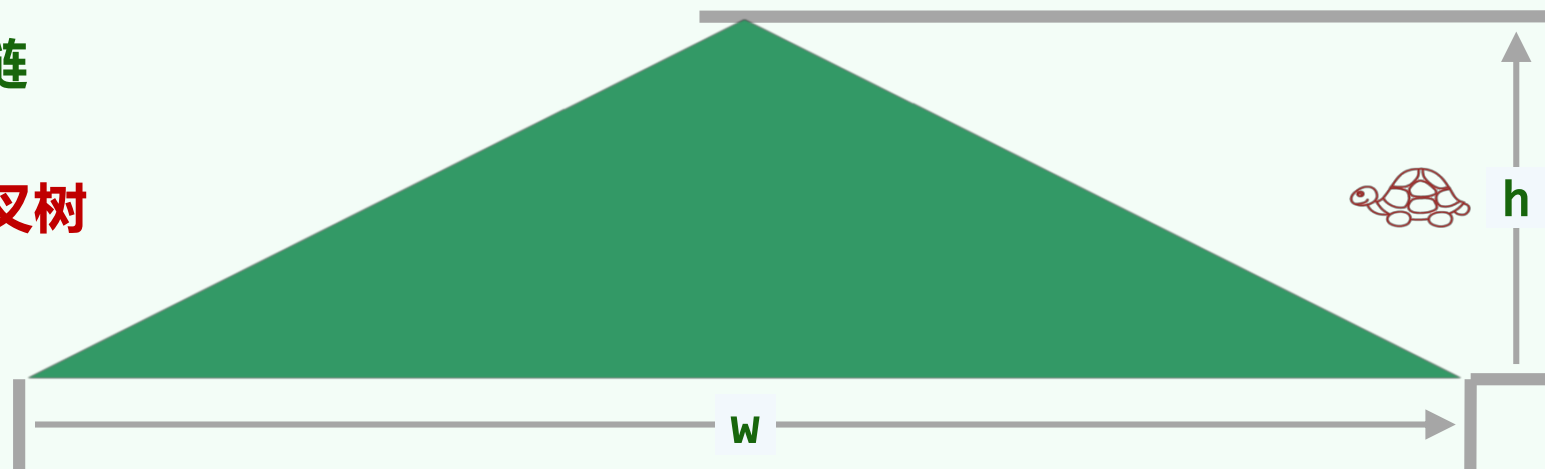
$$h + 1 \leq n \leq 2^{h+1} - 1$$



❖ 特殊情况

- $n = h + 1$: 退化为一条单链
- $n = 2^{h+1} - 1$: 即所谓**满二叉树**

full binary tree



基数

❖ 设度数为0、1和2的节点，各有 n_0 、 n_1 和 n_2 个

❖ 边数 $e = n - 1 = n_1 + 2n_2$

1/2度节点各对应于1/2条入边

❖ 叶节点数 $n_0 = n_2 + 1$

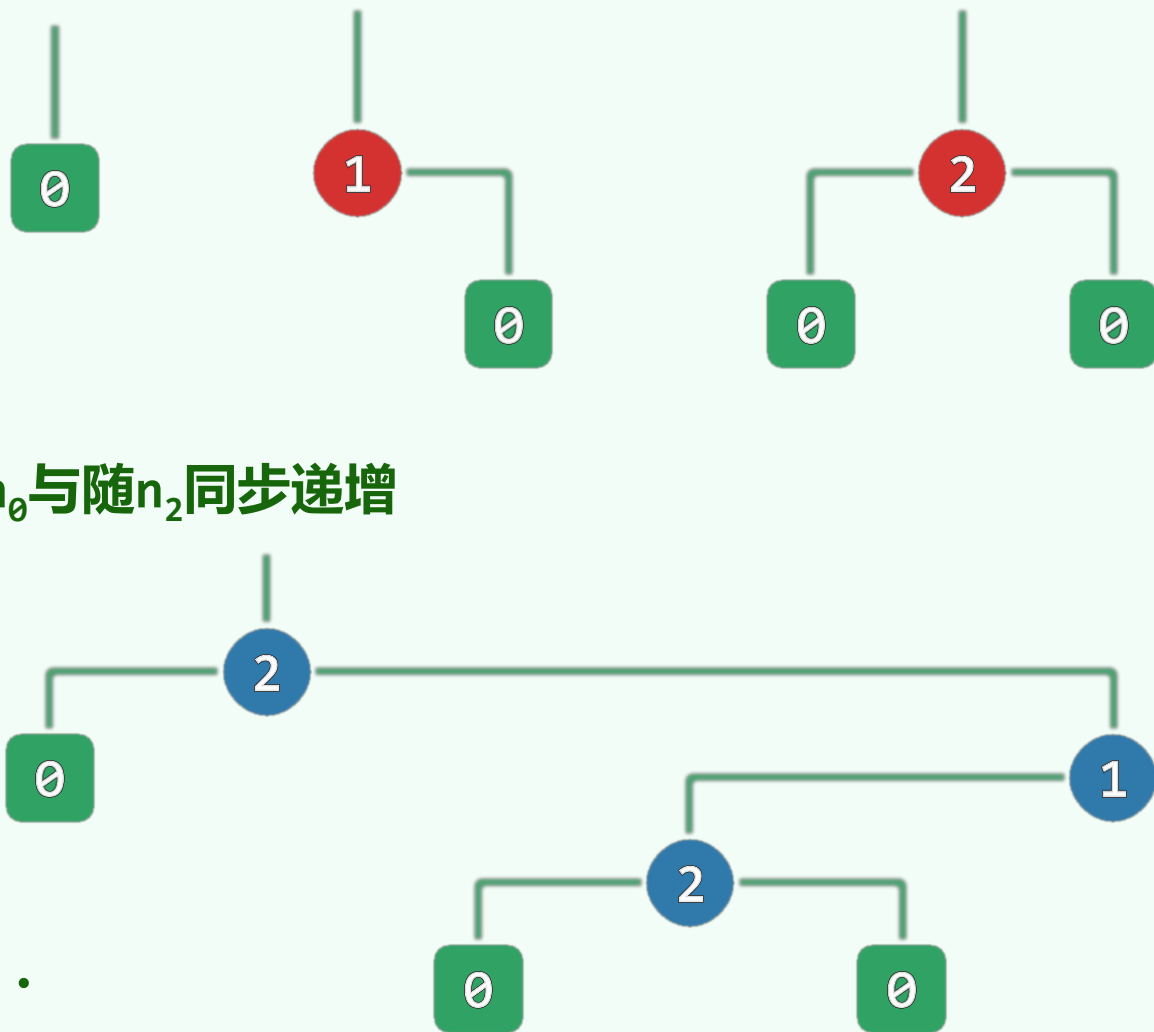
n_1 与 n_0 无关： $h = 0$ 时， $1 = 0 + 1$ ；此后， n_0 与随 n_2 同步递增

❖ 节点数 $n = n_0 + n_1 + n_2 = 1 + n_1 + 2n_2$

❖ 特别地，当 $n_1 = 0$ 时，有

$$e = 2n_2 \text{ 和 } n_0 = n_2 + 1 = (n + 1)/2$$

此时，节点度数均为**偶数**，不含单分支节点...



真二叉树

❖ 通过引入 $n_1 + 2n_0$ 个外部节点

可使原有节点度数统一为2

❖ 如此，即可将任一二叉树

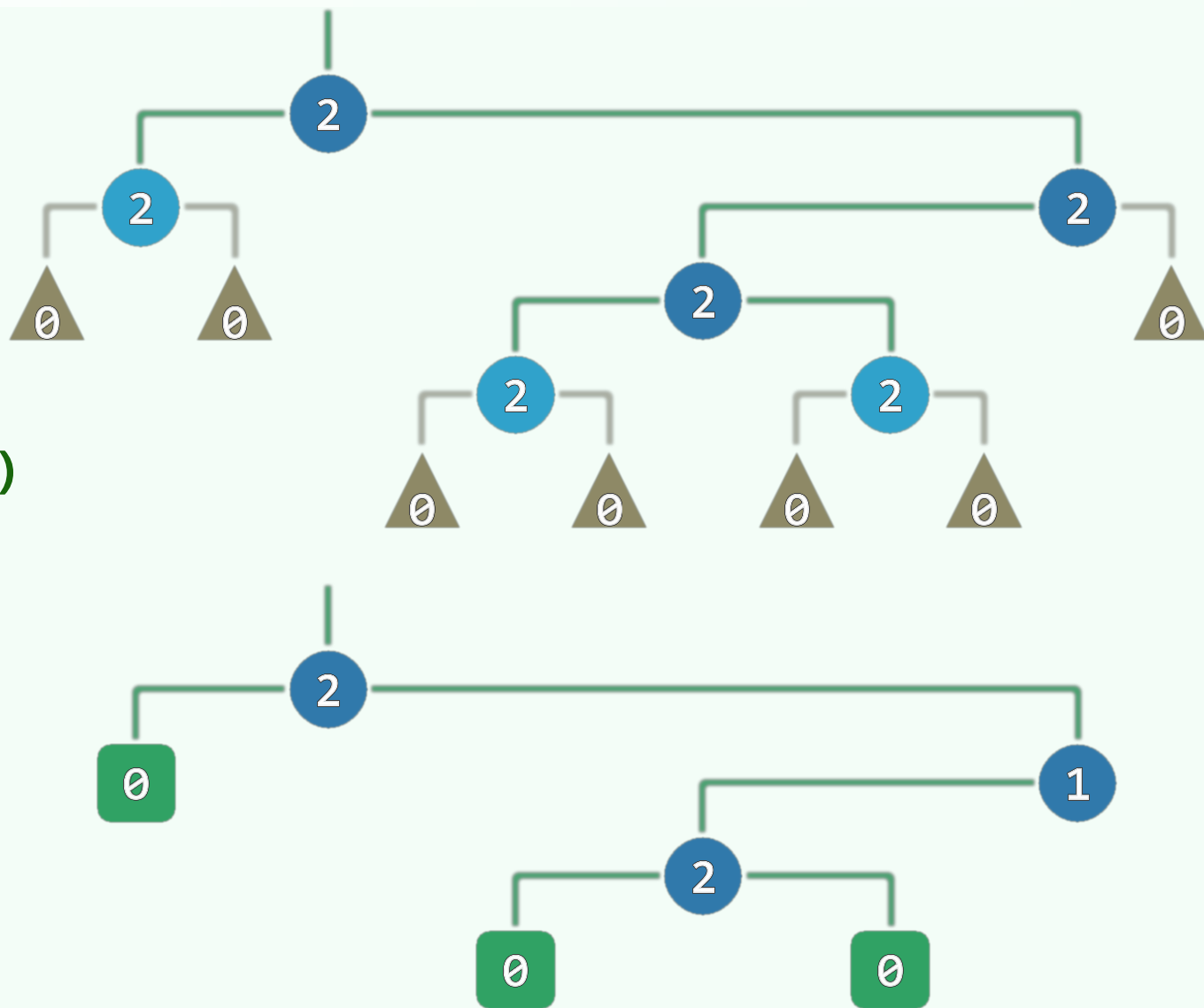
转化为真二叉树 (proper binary tree)

❖ 验证： 如此转换之后，

全树自身的复杂度并未实质增加

❖ 对于红黑树之类的结构，真二叉树

可以简化描述、理解、实现和分析



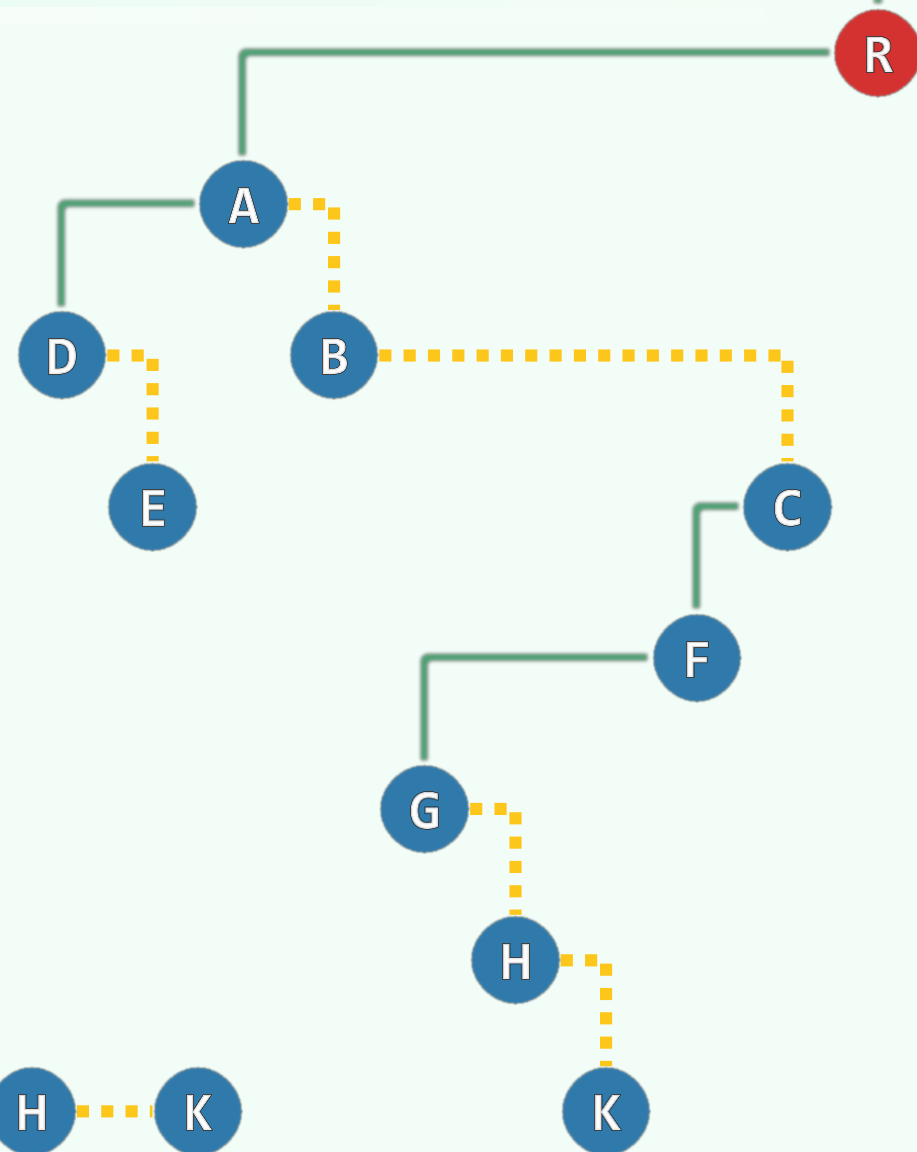
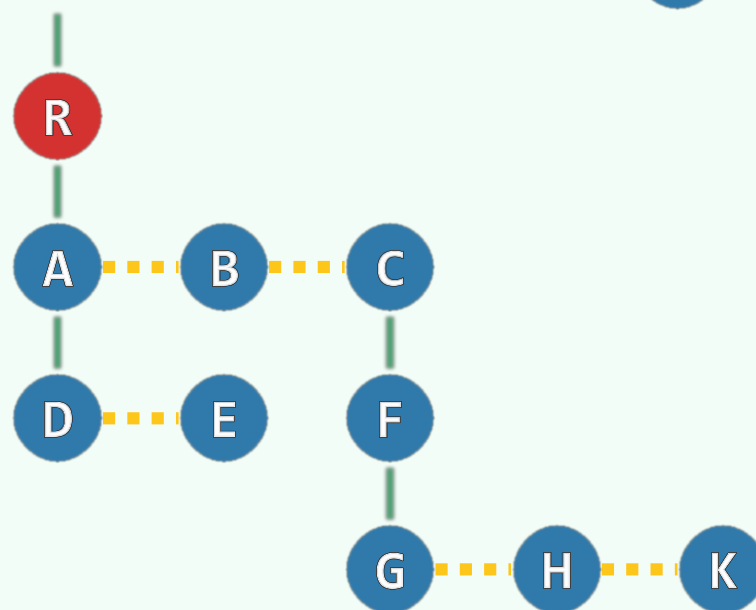
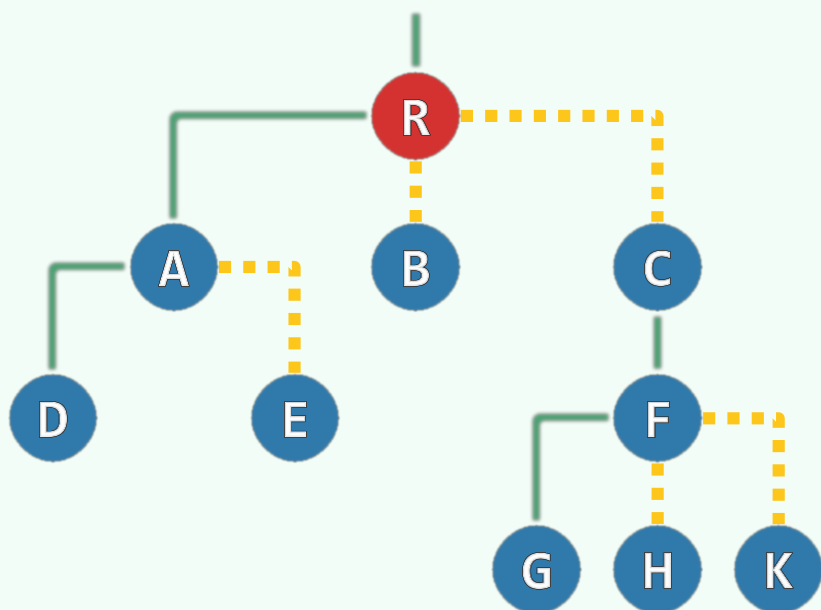
描述多叉树

❖ 有根且有序时的多叉树，均可转化并表示为二叉树

——回忆长子-兄弟表示法...

❖ 长子 ~ 左孩子 `firstChild()` ~ `lc()`

兄弟 ~ 右孩子 `nextSibling()` ~ `rc()`



完全二叉树 ~ 紧凑表示 ~ 以向量实现

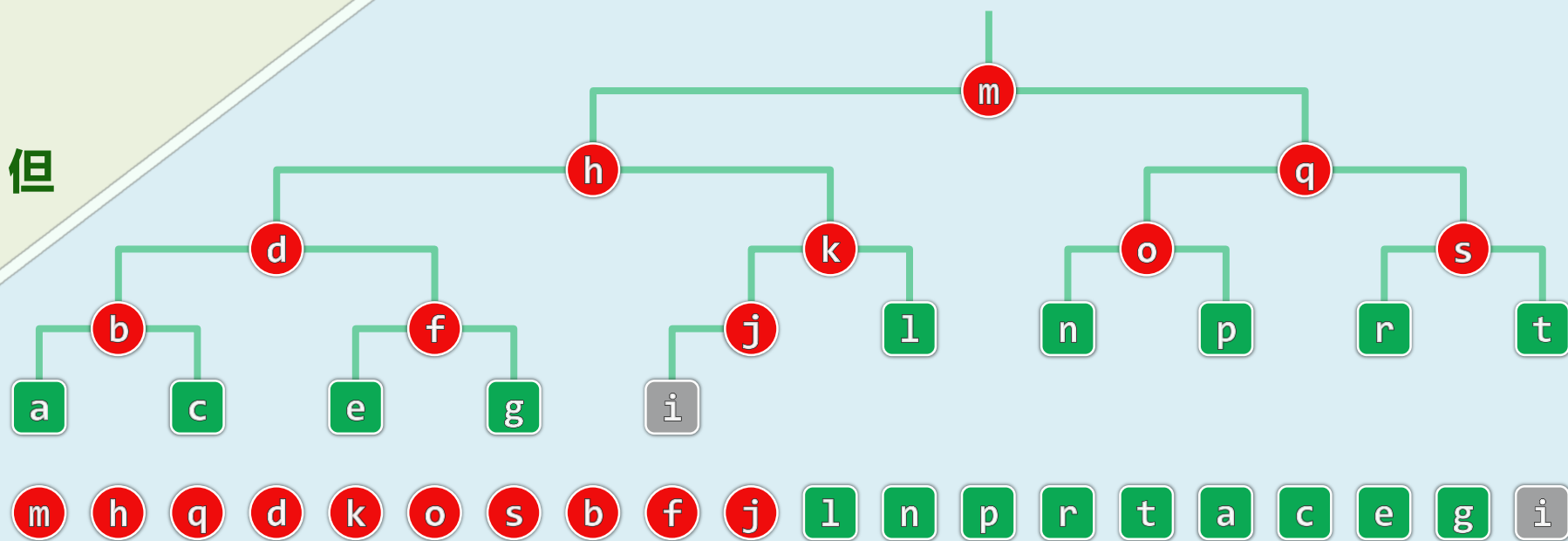
❖ 叶节点仅限于最低两层

底层叶子，均居于次底层叶子左侧（相对于LCA）

除末节点的父亲，内部节点均有双子

❖ 叶节点

- 不致少于内部节点，但
- 至多多出一个



二叉树

二叉树实现

BinNode模板类

```
template <typename T> using BinNodePosi = BinNode<T>*; //节点位置
```

```
template <typename T> struct BinNode {
```

```
    BinNodePosi<T> parent, lc, rc; //父亲、孩子
```

```
    T data; int height; int size(); //高度、子树规模
```

```
    BinNodePosi<T> insertAsLC( T const & ); //作为左孩子插入新节点
```

```
    BinNodePosi<T> insertAsRC( T const & ); //作为右孩子插入新节点
```

```
    BinNodePosi<T> succ(); // (中序遍历意义下) 当前节点的直接后继
```

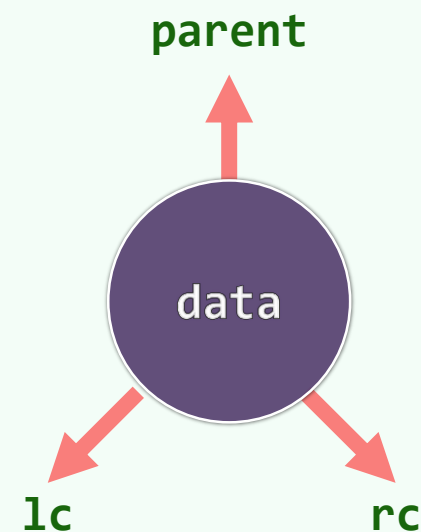
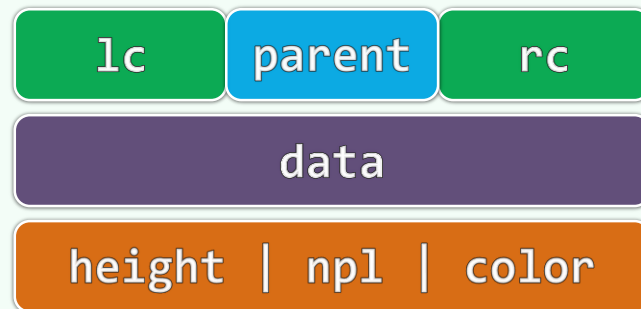
```
    template <typename VST> void travLevel( VST & ); //子树层次遍历
```

```
    template <typename VST> void travPre( VST & ); //子树先序遍历
```

```
    template <typename VST> void travIn( VST & ); //子树中序遍历
```

```
    template <typename VST> void travPost( VST & ); //子树后序遍历
```

```
};
```

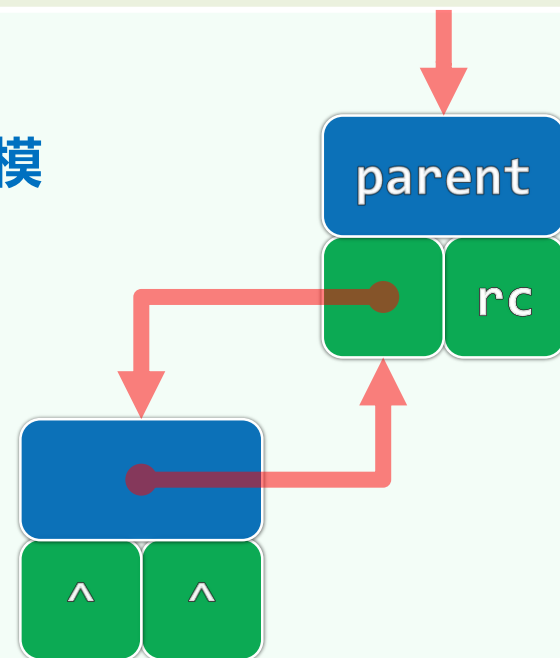


BinNode接口实现

```
❖ template <typename T> BinNodePosi<T> BinNode<T>::insertAsLC( T const & e )  
    { return lc = new BinNode( e, this ); }
```

```
❖ template <typename T> BinNodePosi<T> BinNode<T>::insertAsRC( T const & e )  
    { return rc = new BinNode( e, this ); }
```

```
❖ template <typename T>  
    int BinNode<T>::size() { //后代总数，亦即以其为根的子树的规模  
        int s = 1; //计入本身  
        if (lc) s += lc->size(); //递归计入左子树规模  
        if (rc) s += rc->size(); //递归计入右子树规模  
        return s;  
    } //O( n = |size| )
```



BinTree模板类

```
template <typename T> class BinTree {  
protected: int _size; //规模  
            BinNodePosi<T> _root; //根节点  
            virtual int updateHeight( BinNodePosi<T> x ); //更新节点x的高度  
            void updateHeightAbove( BinNodePosi<T> x ); //更新x及祖先的高度  
public:      int size() const { return _size; } //规模  
            bool empty() const { return !_root; } //判空  
            BinNodePosi<T> root() const { return _root; } //树根  
            /* ... 子树接入、删除和分离接口; 遍历接口 ... */  
}
```

高度更新

```
#define stature(p) ( (p) ? (p)->height : -1 ) //节点高度——空树 ~ -1
```

```
template <typename T> //更新节点x高度，具体规则因树不同而异
```

```
int BinTree<T>::updateHeight( BinNodePosi<T> x ) //此处采用常规二叉树规则， $O(1)$ 
```

```
{ return x->height = 1 + max( stature( x->lc ), stature( x->rc ) ); }
```

```
template <typename T> //更新节点及其历代祖先的高度
```

```
void BinTree<T>::updateHeightAbove( BinNodePosi<T> x ) // $O(n = \text{depth}(x))$ 
```

```
{ while (x) { updateHeight(x); x = x->parent; } } //可优化
```

节点插入

BinNodePosi<T> BinTree<T>::insert(BinNodePosi<T> x, T const & e); //作为右孩子

BinNodePosi<T> BinTree<T>::insert(T const & e, BinNodePosi<T> x) { //作为左孩子

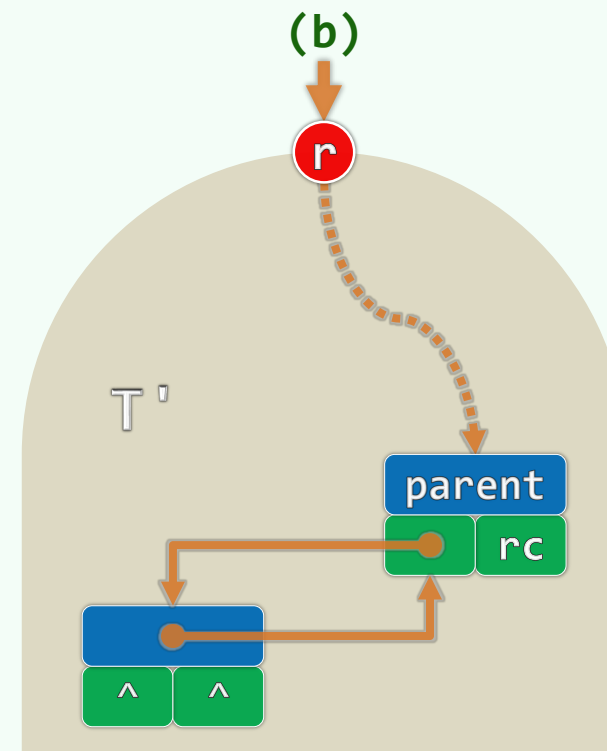
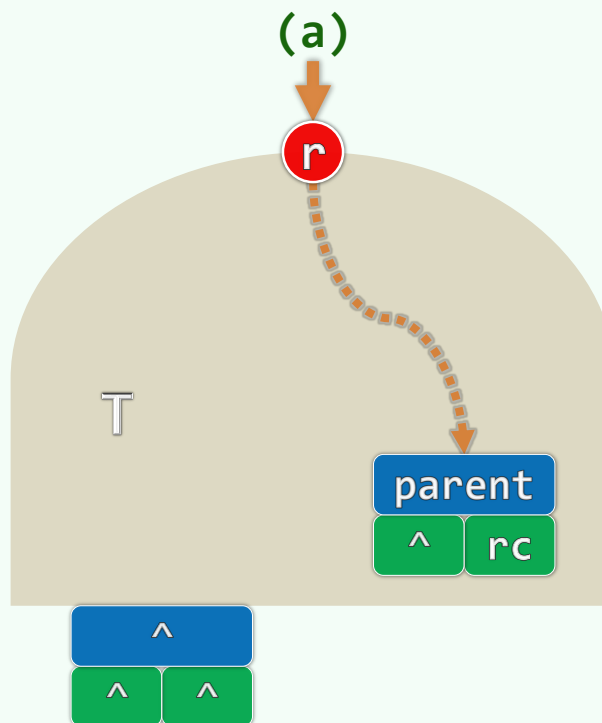
_size++;

x->insertAsLC(e);

updateHeightAbove(x);

return x->lc;

}

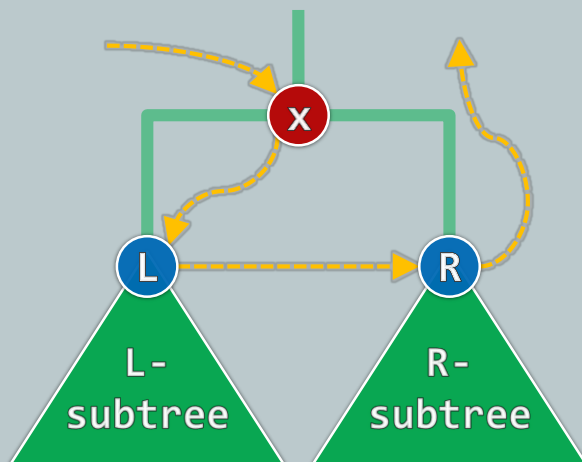


二叉树 先序遍历

遍历

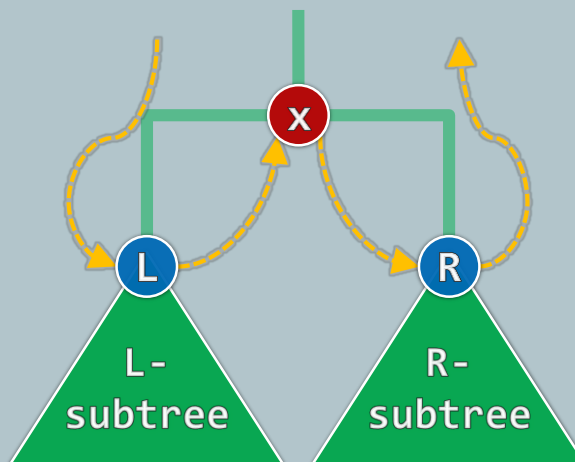
❖ 按照**某种次序**访问树中各节点，每个节点被访问**恰好一次**： $\boxed{T} = \boxed{L} \cup \boxed{x} \cup \boxed{R}$

先序
preorder



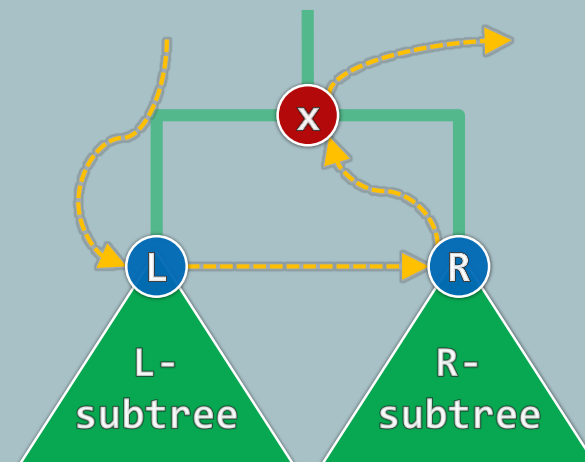
$\boxed{x} \mid \boxed{L} \mid \boxed{R}$

中序
inorder



$\boxed{L} \mid \boxed{x} \mid \boxed{R}$

后序
postorder



$\boxed{L} \mid \boxed{R} \mid \boxed{x}$

❖ 遍历结果 ~ 遍历过程 ~ 遍历次序 ~ 遍历策略

递归实现

❖ 应用：先序输出文件树结构： `c:\> tree.com c:\windows`

❖ `template <typename T, typename VST>`

```
void traverse( BinNodePosi<T> x, VST & visit ) {
```

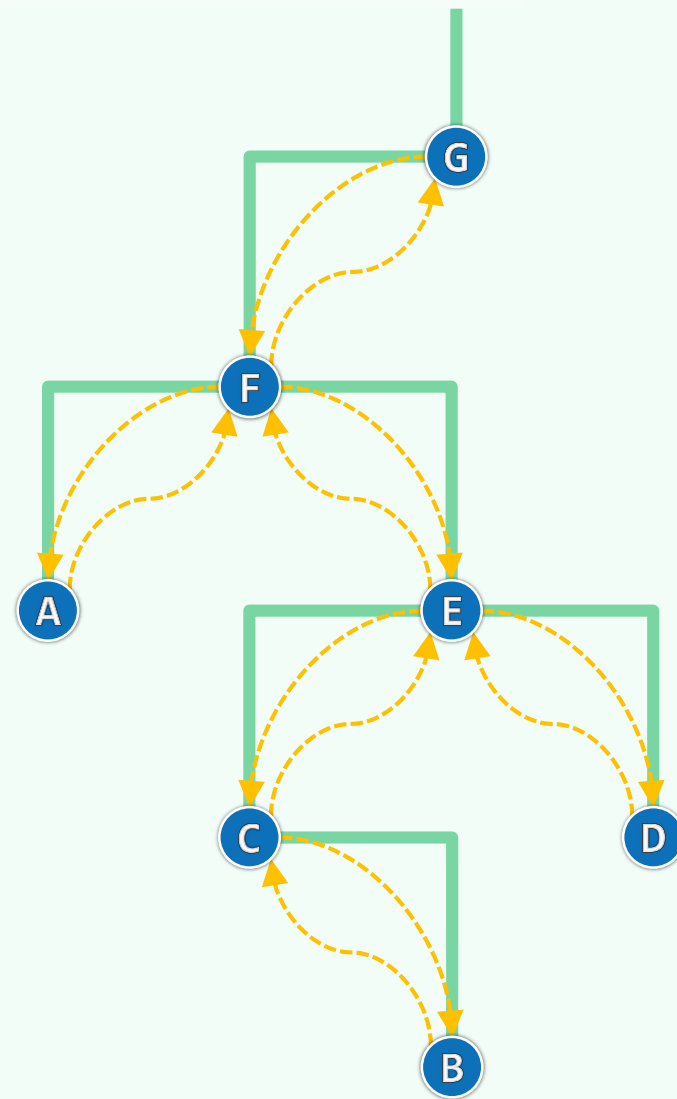
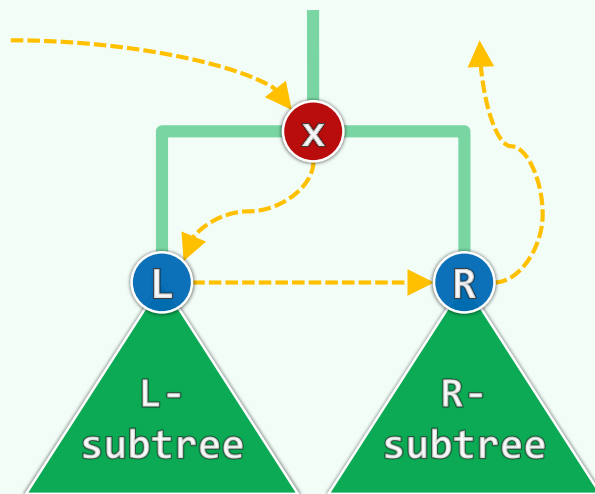
```
    if ( ! x ) return;
```

```
    visit( x->data );
```

```
    traverse( x->lc, visit );
```

```
    traverse( x->rc, visit );
```

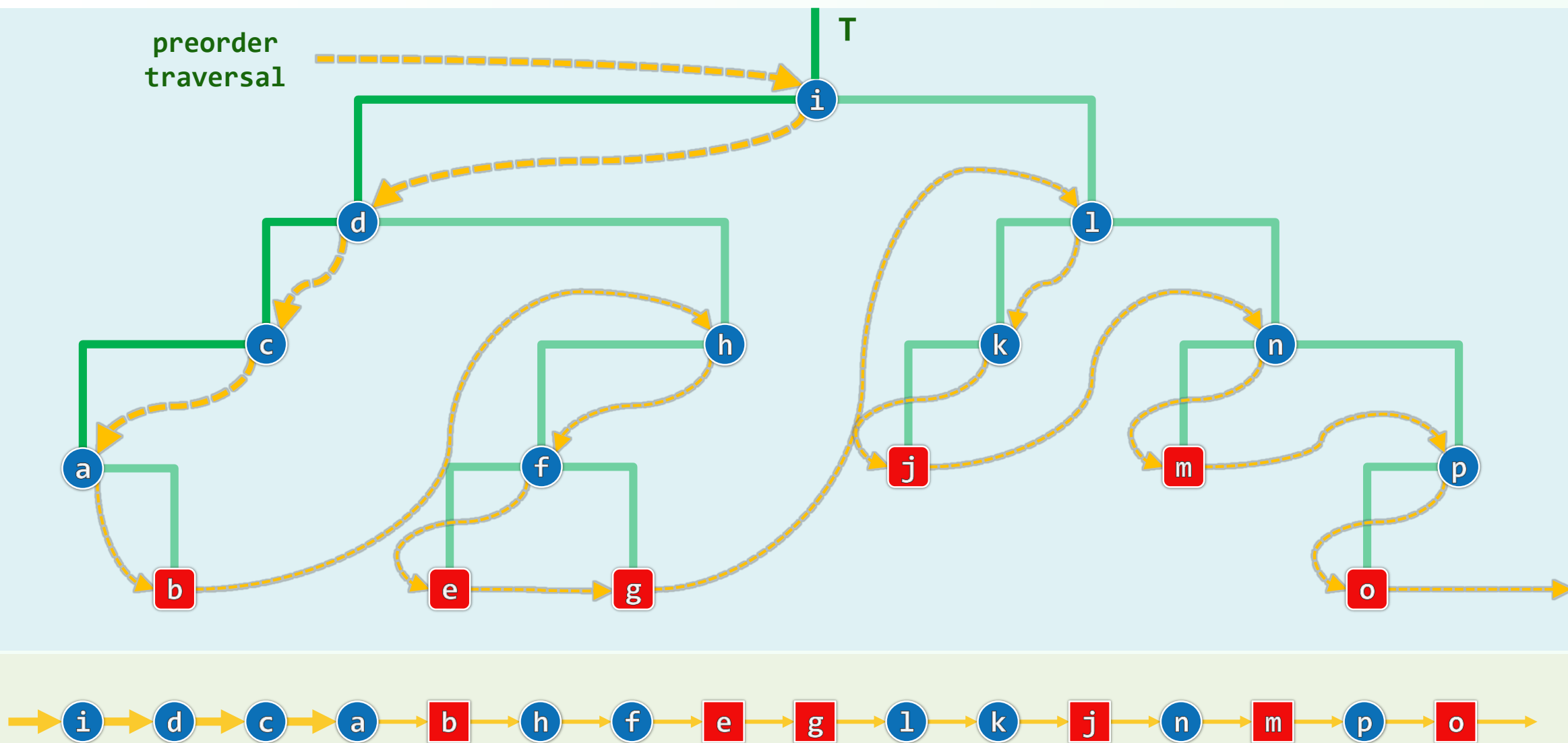
```
} //O(n)
```



❖ 制约：使用**默认**的Call Stack，允许的递归**深度**有限

❖ 挑战：不依赖递归机制，能否实现先序遍历？如何实现？效率如何？

观察



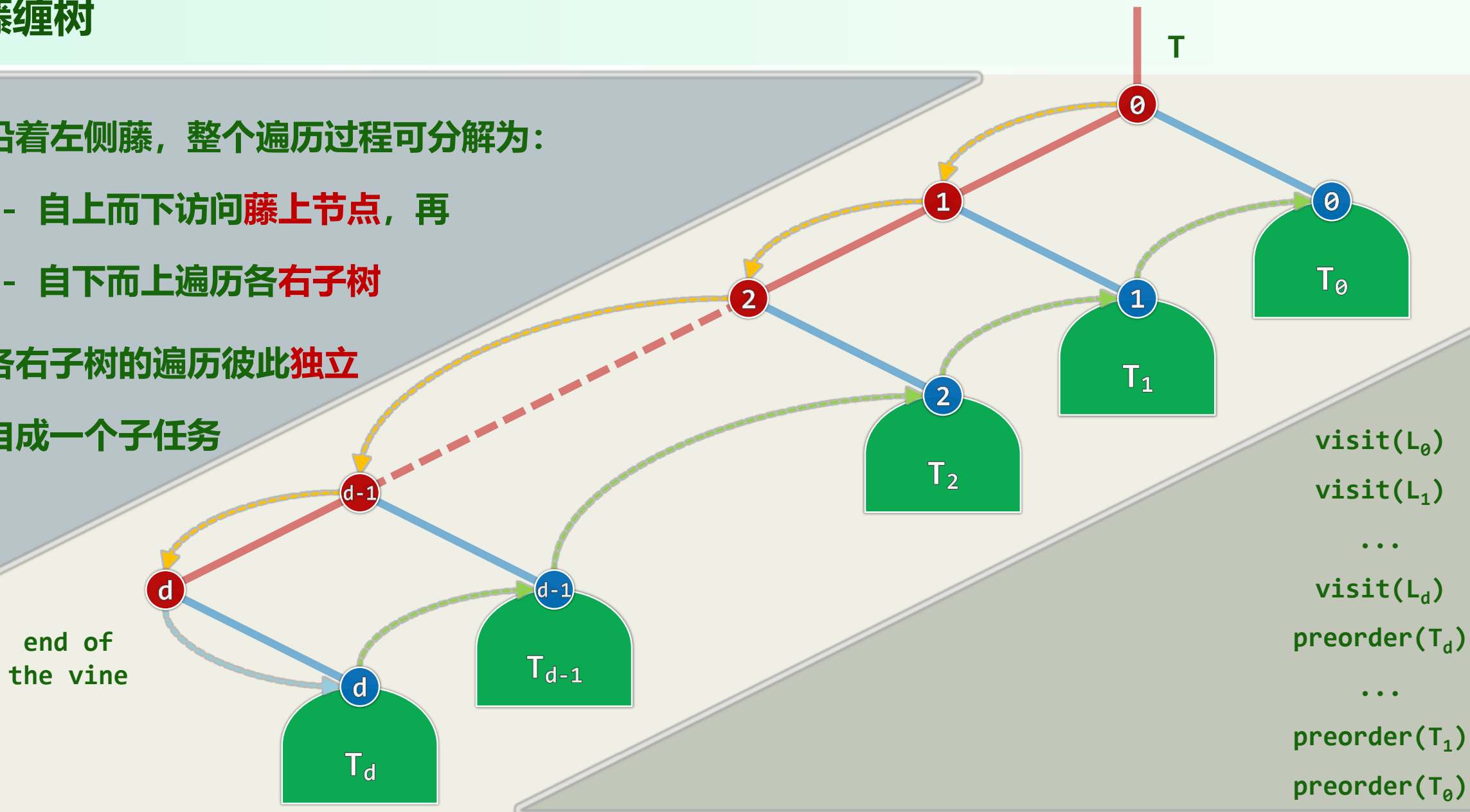
藤缠树

❖ 沿着左侧藤，整个遍历过程可分解为：

- 自上而下访问藤上节点，再
- 自下而上遍历各右子树

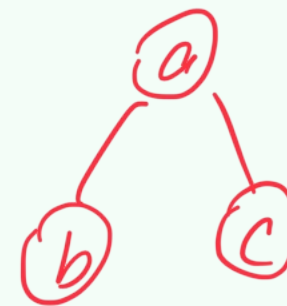
❖ 各右子树的遍历彼此独立

自成一个子任务



序曲

```
❖ template <typename T, typename VST> static void visitAlongVine  
( BinNodePosi<T> x, VST & visit, Stack < BinNodePosi<T> > & S ) { //分摊 $O(1)$   
  
    while ( x ) { //反复地  
  
        visit( x->data ); //访问当前节点 $\alpha$   
  
        S.push( x->rc ); //右孩子C (右子树) 入栈 (将来逆序出栈)  
  
        x = x->lc; //沿藤下行b  
  
    } //只有右孩子、NULL可能入栈—增加判断以剔除后者, 是否值得?  
  
}
```



全曲

❖ template <typename T, typename VST>

```
void travPre_I2( BinNodePosi<T> x, VST & visit ) {
```

```
    Stack < BinNodePosi<T> > S; //辅助栈
```

```
    while ( true ) { //以右子树为单位，逐批访问节点
```

```
        visitAlongVine( x, visit, S ); //访问子树x的藤蔓，各右子树（根）入栈缓冲
```

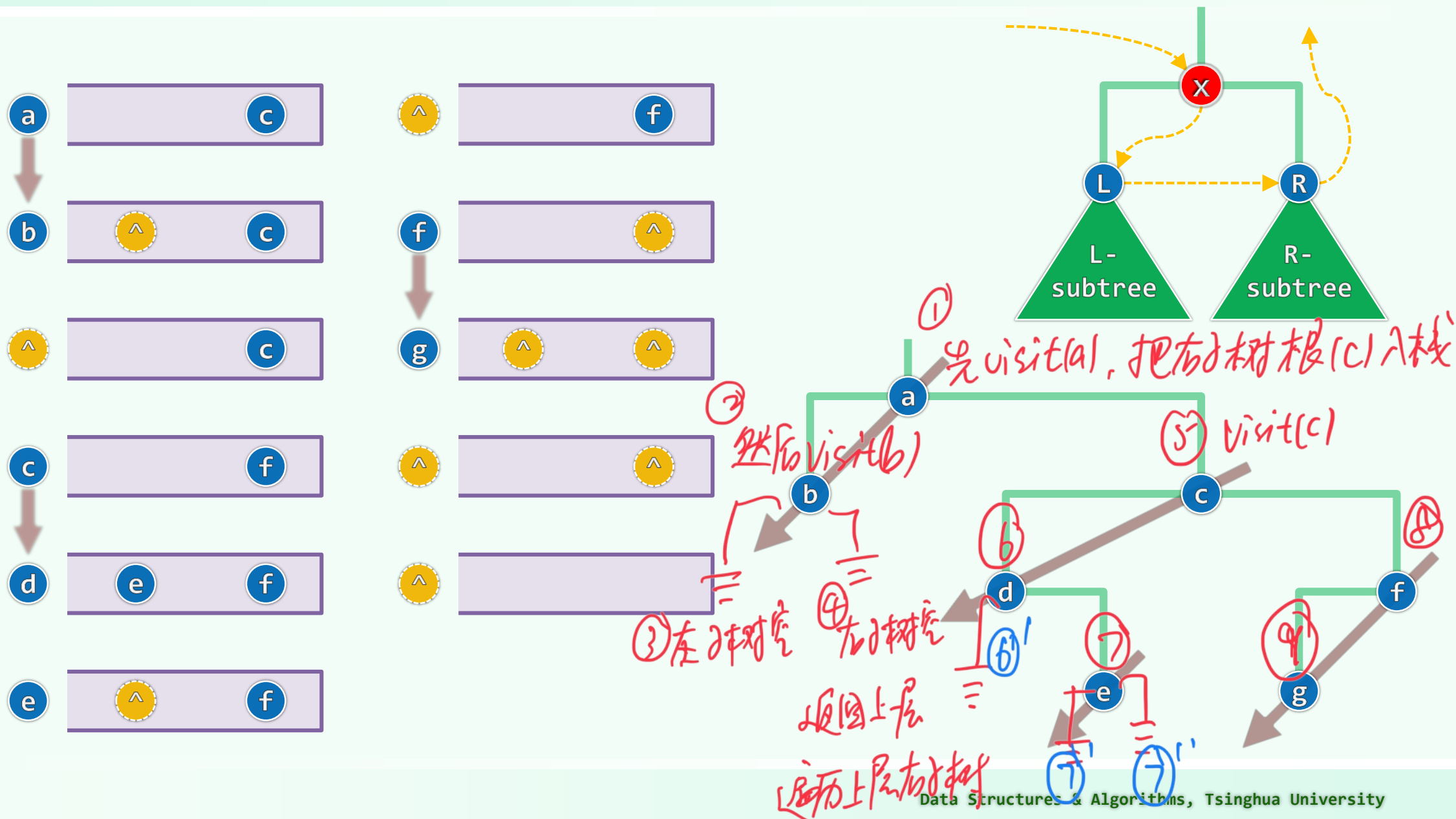
```
        if ( S.empty() ) break; //栈空即退出 没有右子树
```

```
        x = S.pop(); //弹出下一右子树（根） 继续返回上一层
```

```
    } // #pop = #push = #visit =  $O(n)$  = 分摊 $O(1)$ 
```

```
}
```

实例



二叉树 中序遍历

递归实现

❖ 应用：中序输出文件树结构： `printBinTree()`

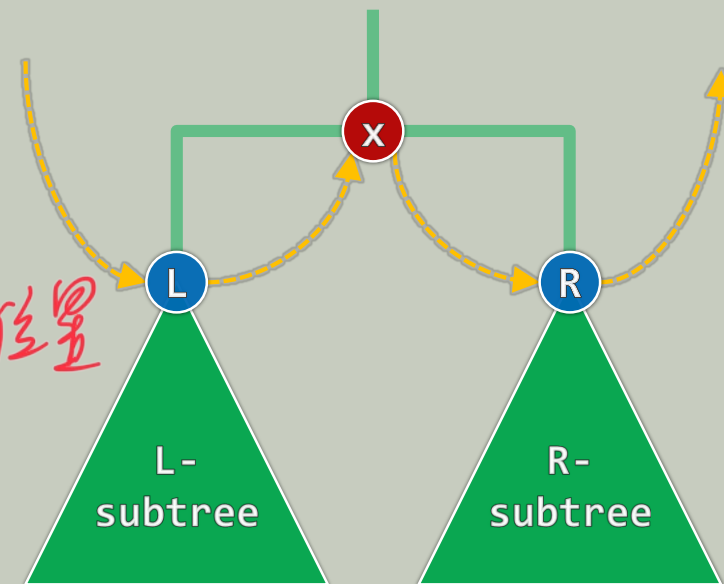
❖ `template <typename T, typename VST>`

```
void traverse( BinNodePosi<T> x, VST & visit ) {  
    if ( !x ) return;  
    traverse( x->lc, visit );  
    visit( x->data );  
    traverse( x->rc, visit ); //tail  
}
```

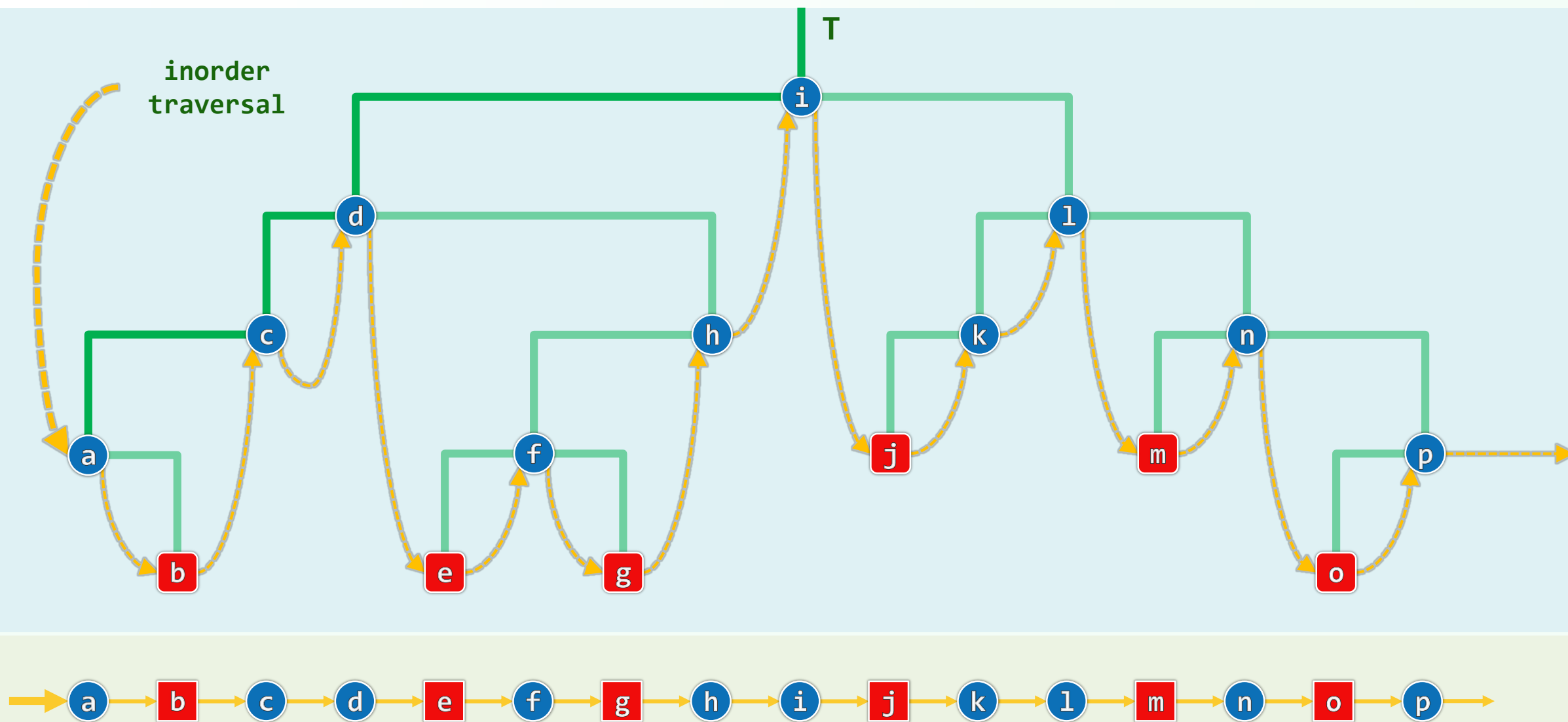
❖ $T(n) = T(a) + \mathcal{O}(1) + T(n - a - 1) = \mathcal{O}(n)$

❖ 挑战：不依赖递归机制，如何实现中序遍历？效率如何？

交换遍历与访问的位置



观察



藤缠树

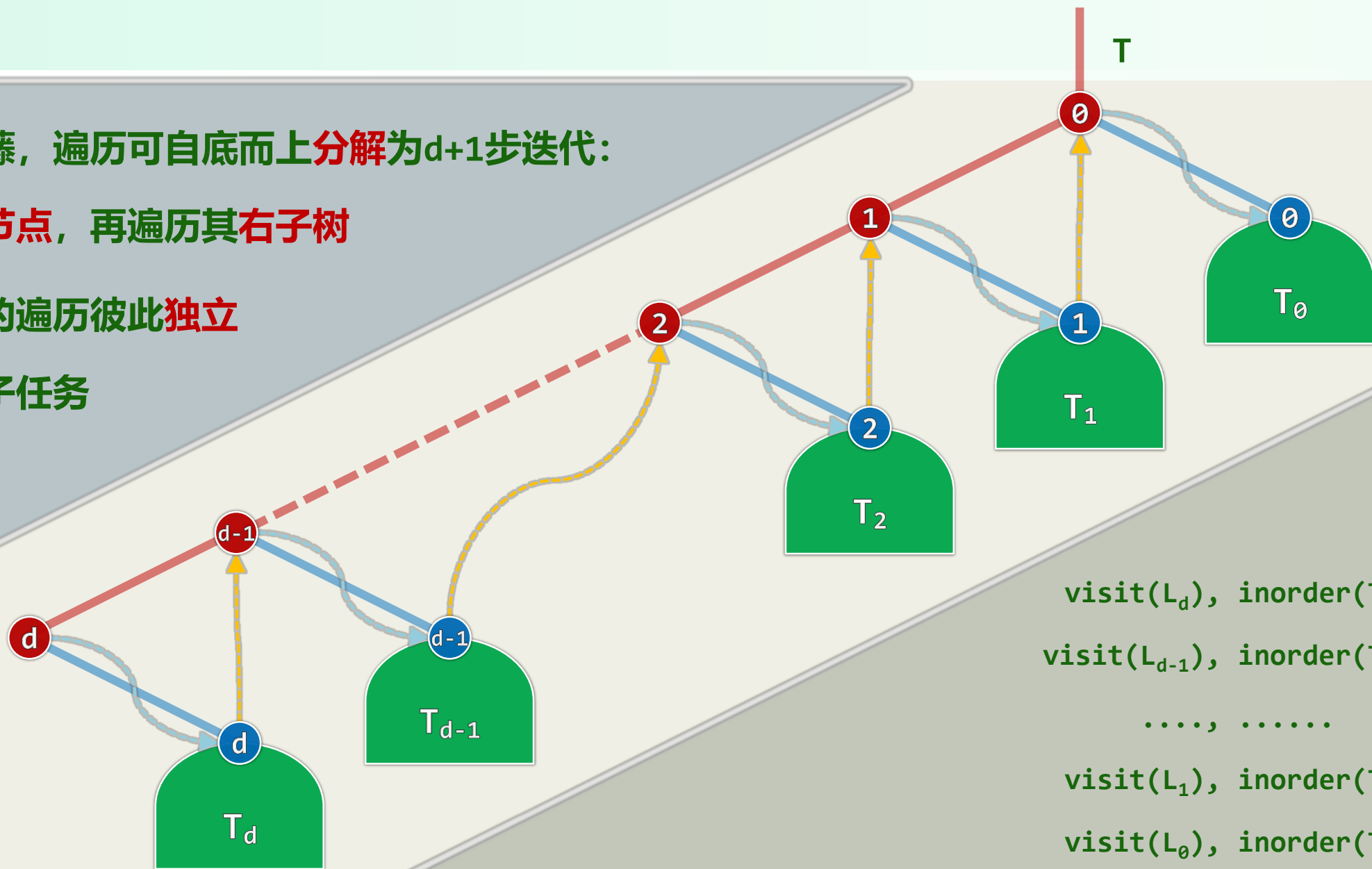
❖ 沿着左侧藤，遍历可自底而上分解为 $d+1$ 步迭代：

访问藤上节点，再遍历其右子树

❖ 各右子树的遍历彼此独立

自成一个子任务

end of
the vine



$\text{visit}(L_d), \text{inorder}(T_d)$
 $\text{visit}(L_{d-1}), \text{inorder}(T_{d-1})$
.....
 $\text{visit}(L_1), \text{inorder}(T_1)$
 $\text{visit}(L_0), \text{inorder}(T_0)$

序曲

□ 14
输入

门轴x

1. TP (2)

25.5

$$y + x = 12 \neq 2$$

```
template <typename T> static void goAlongVine(
```

```
BinNodePosi<T> x,
```

```
Stack < BinNodePosi<T> > & S
```

)

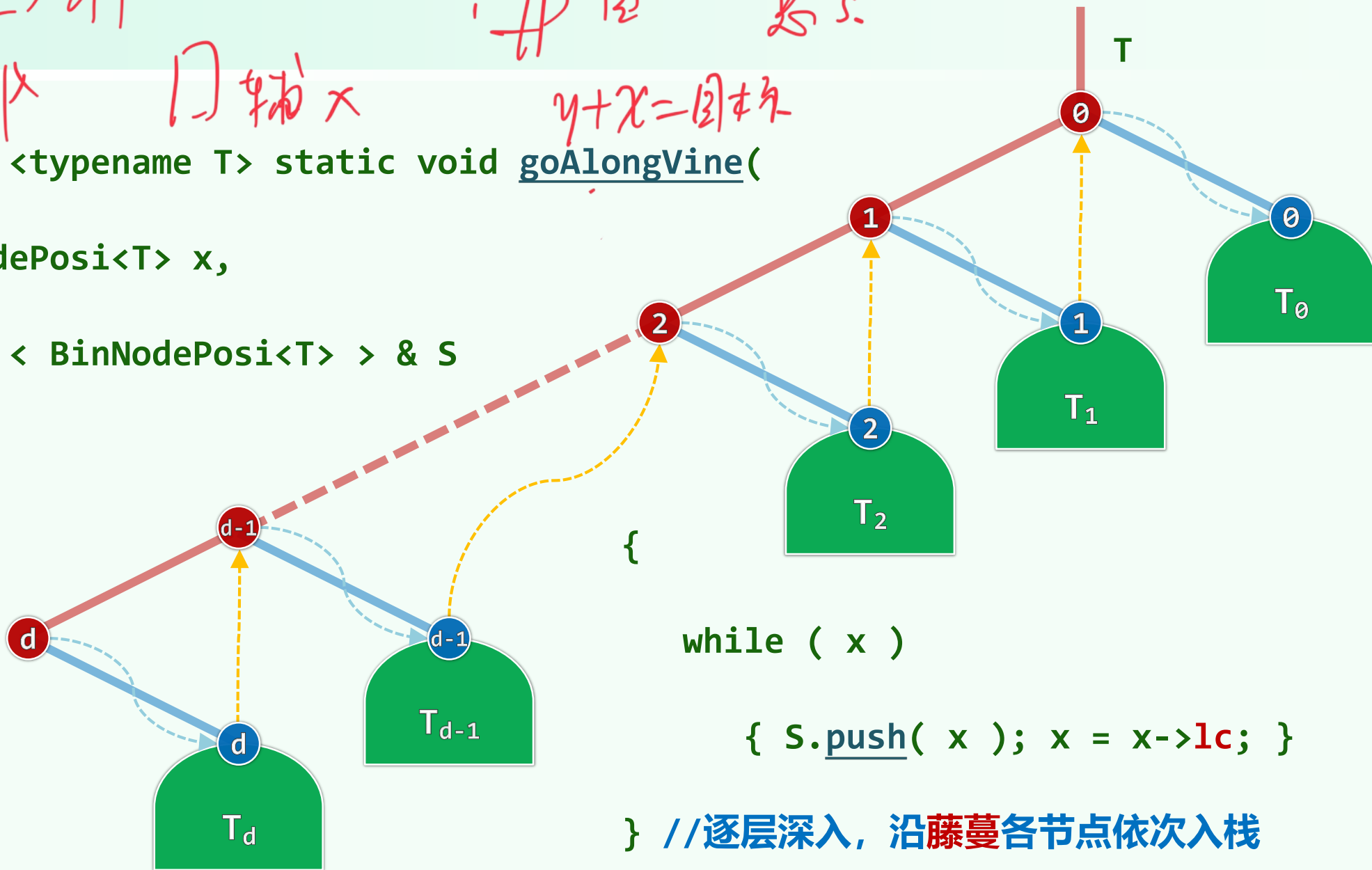
end of
the vine

{

```
while ( x )
```

```
{ S.push( x ); x = x->lc; }
```

```
//逐层深入，沿藤蔓各节点依次入栈
```



全曲

❖ template <typename T, typename V>

```
void travIn_I1( BinNodePosi<T> x, V& visit ) {
```

```
    Stack < BinNodePosi<T> > S; //辅助栈
```

```
    while ( true ) { //反复地
```

```
        goAlongVine( x, S ); //从当前节点出发，逐批入栈
```

```
        if ( S.empty() ) break; //直至所有节点处理完毕
```

```
        x = S.pop(); //x的左子树或为空，或已遍历（等效于空），故可以
```

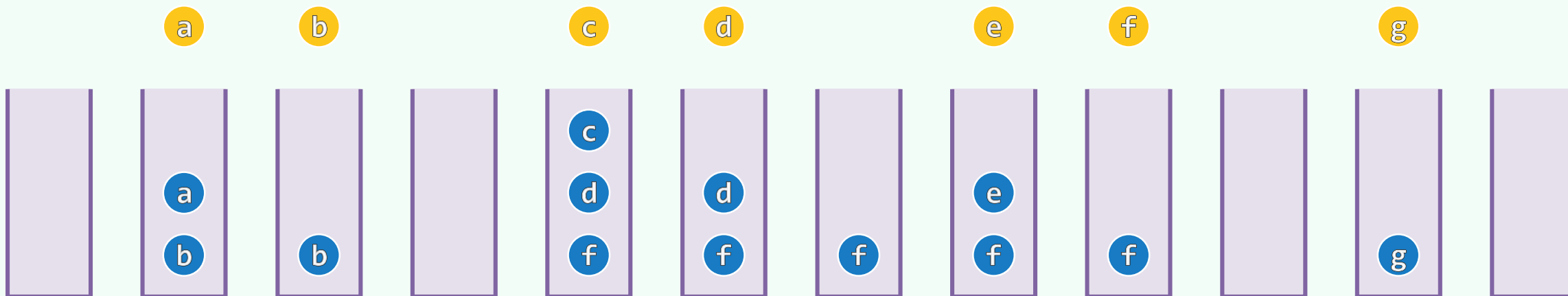
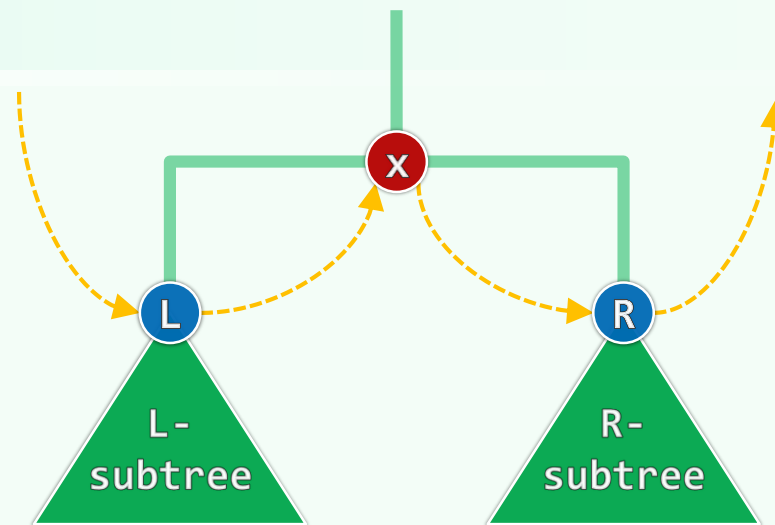
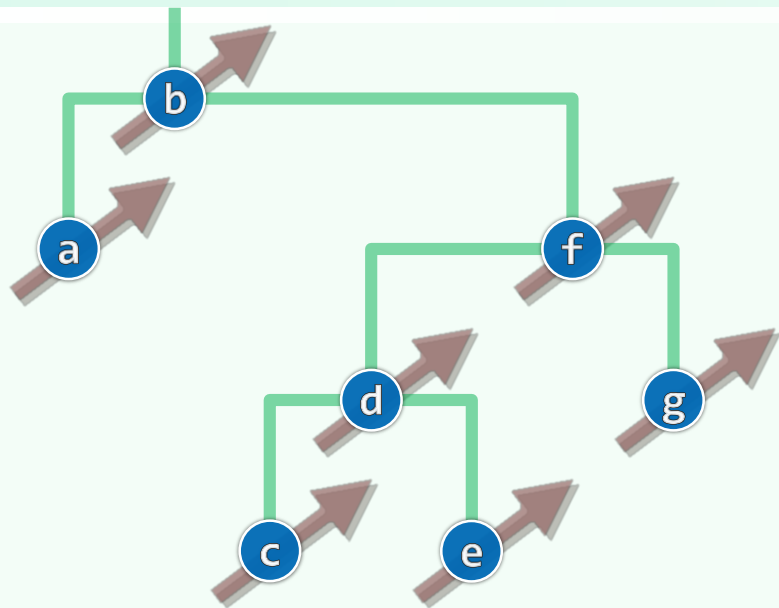
```
        visit( x->data ); //立即访问之
```

```
        x = x->rc; //再转向其右子树（可能为空，留意处理手法）
```

```
    }
```

```
}
```

实例



二叉树 中序遍历：分析

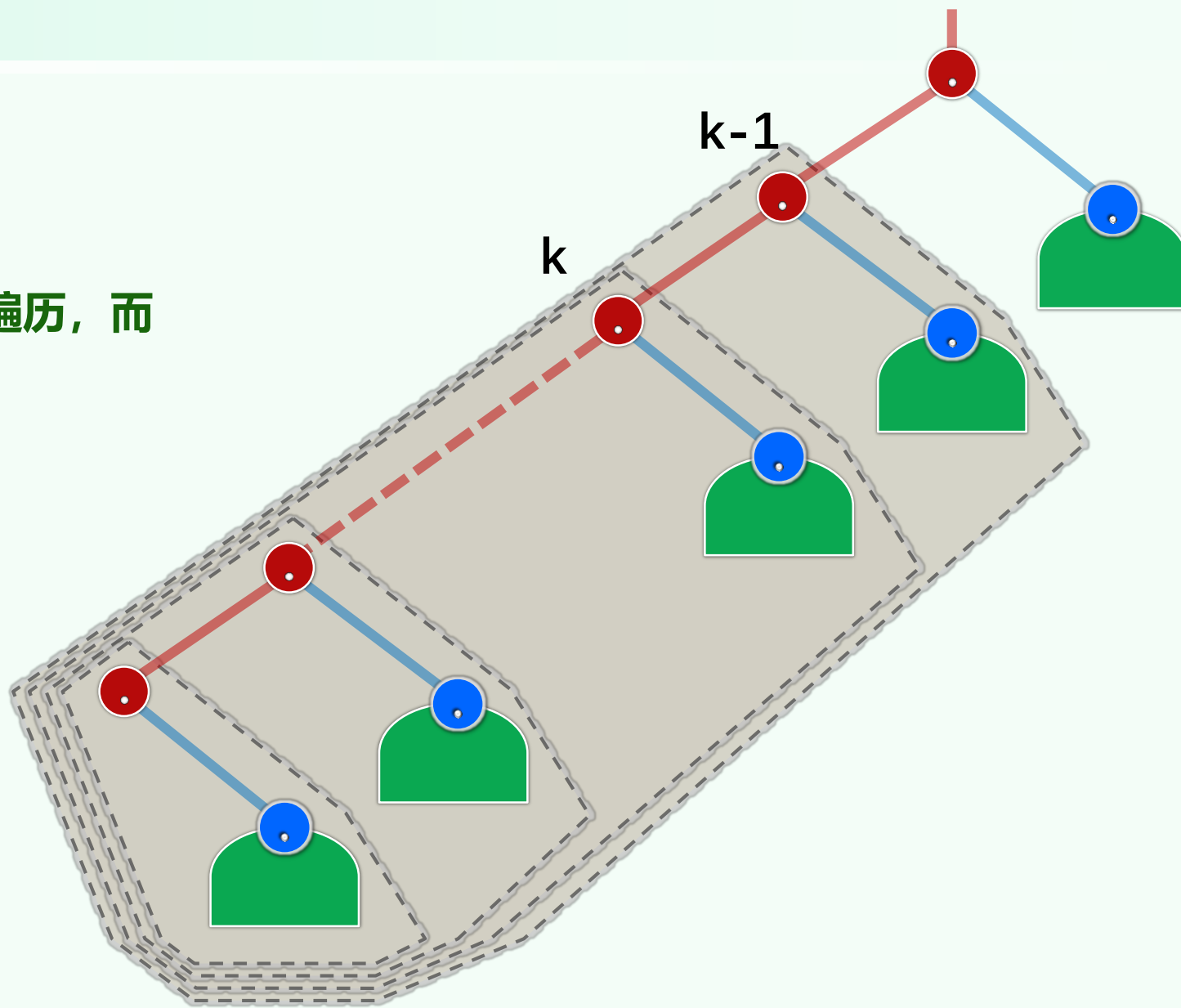
正确性：数学归纳

❖ 每个节点出栈时

- 其左子树或不存在，或已完全遍历，而
- 右子树尚未入栈

❖ 于是，每当有节点出栈，只需

- 访问它，然后
- 从其右孩子出发...



效率：分摊分析

❖ 是否 $O(n)$ ，取决于以下条件

- 每次迭代，都**恰有**一个节点出栈并被访问 //满足
- 每个节点入栈一次且仅一次 //满足
- 每次迭代只需 $O(1)$ 时间 //不再满足，因为...

❖ 单次调用`goAlongVine()`就可能需做 $\Omega(n)$ 次入栈操作，共需 $\Omega(n)$ 时间

❖ 既然如此，难道总体将需要... $O(n^2)$ 时间？

❖ 事实上，这个界远远不紧...

请利用**分摊**原理，自行分析 //Aggregate

❖ 更多的实现：`travIn_I2()` + `travIn_I3()` + `travIn_I4()`

二叉树

中序遍历：后继和前驱

直接后继：右后代

//在中序遍历意义下的直接后继

//稍后将被BST::remove中的removeAt()调用

```
template <typename T>
```

```
BinNodePosi<T> BinNode<T>::succ() {
```

```
    BinNodePosi<T> s = this;
```

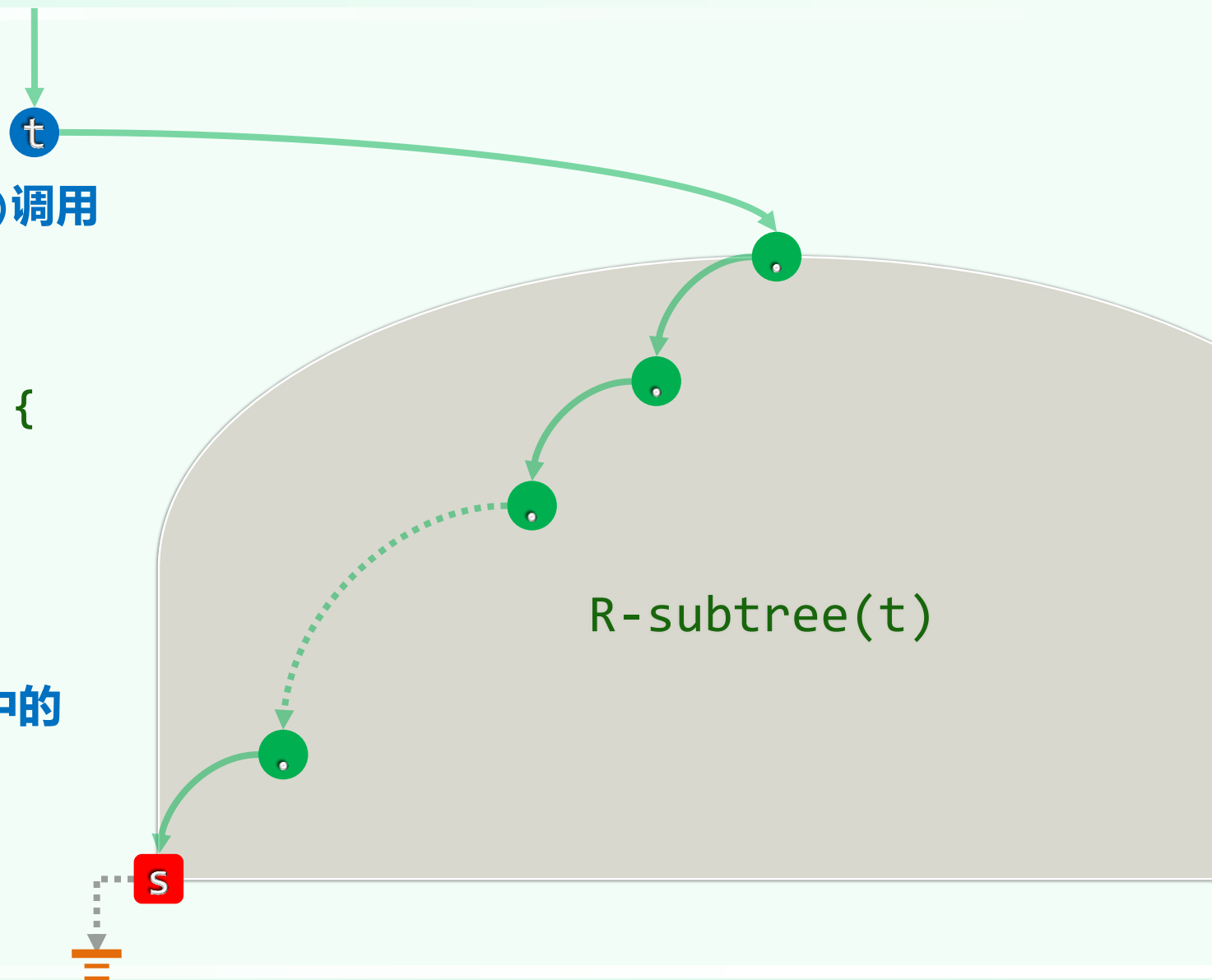
```
    if ( rc ) { //若有右孩子, 则
```

```
        s = rc; //直接后继必是右子树中的
```

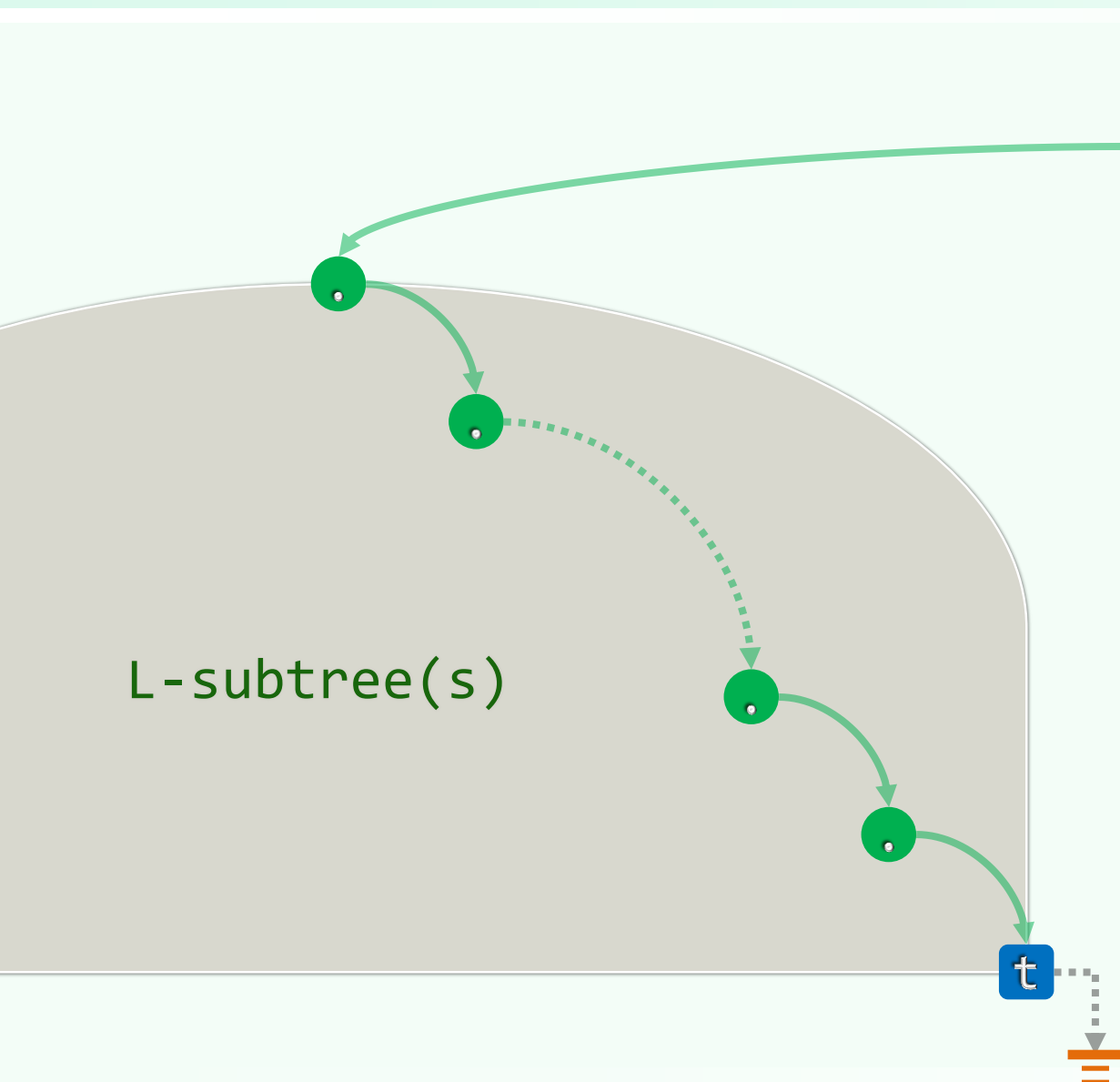
```
        while ( HasLChild( * s ) )
```

```
            s = s->lc; //最小节点
```

```
    } else /* ... */
```



直接后继：左父亲



```
} else { //否则
```

//后继应是“以当前节点为直接前驱者”

```
while ( IsRChild( * s ) )
```

s = s->parent; //不断朝左上移动

//最后再朝右上移动一步

```
s = s->parent; //可能是NULL
```

```
}
```

```
return s; //两种情况下，运行时间分别为
```

```
} //当前节点的高度与深度，不过 $O(h)$ 
```

二叉树 后序遍历

递归实现

❖ 应用: BinNode::size() + BinTree::updateHeight()

❖ `template <typename T, typename VST>`

```
void traverse( BinNodePosi<T> x, VST & visit ) {
```

```
    if ( ! x ) return;
```

```
    traverse( x->lc, visit );
```

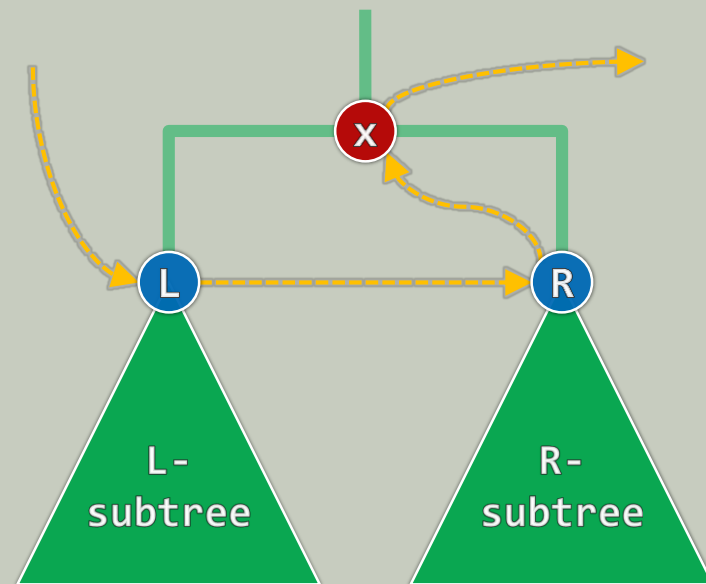
```
    traverse( x->rc, visit );
```

```
    visit( x->data );
```

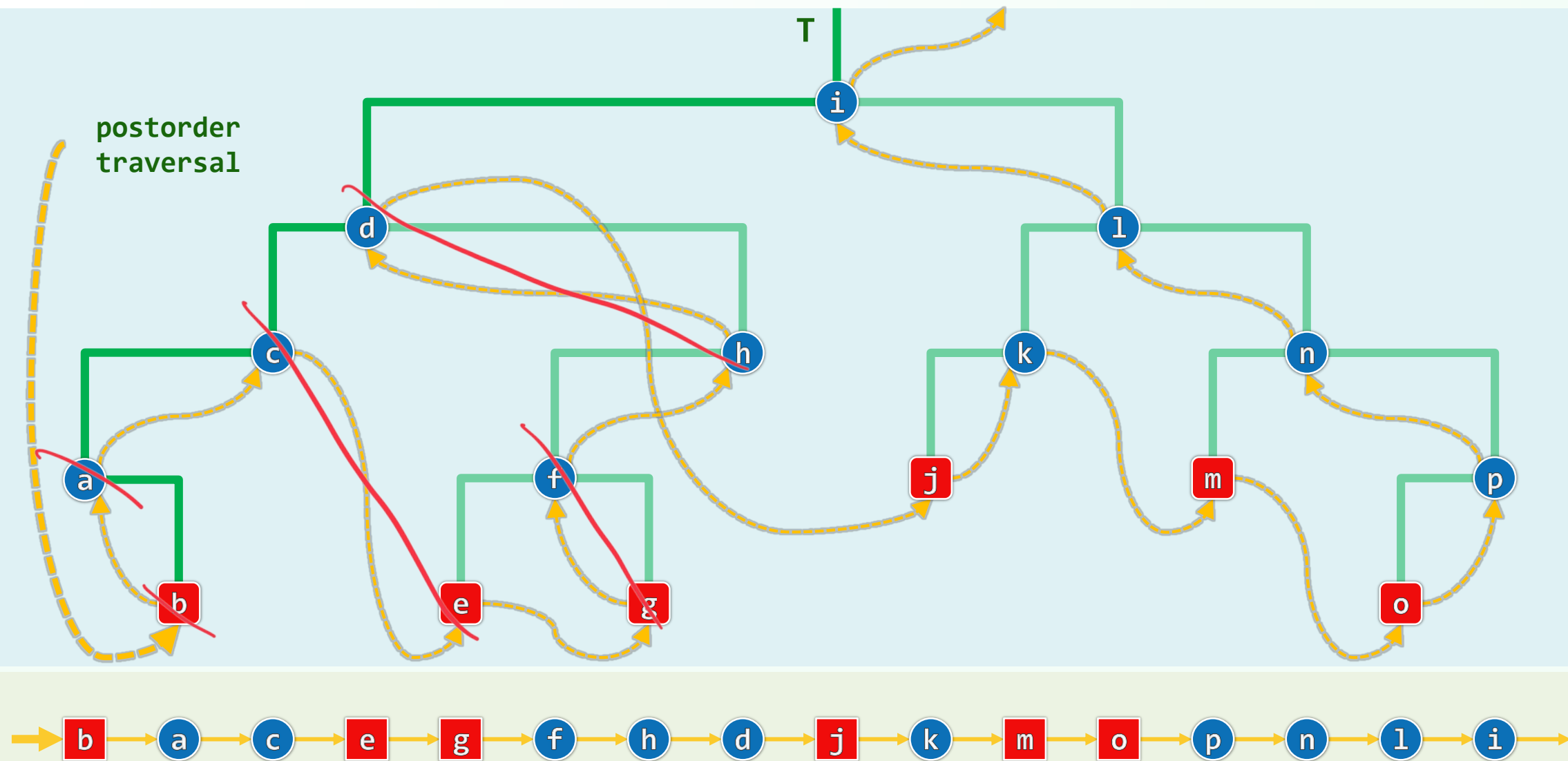
```
}
```

❖ $T(n) = T(a) + T(n - a - 1) + \mathcal{O}(1) = \mathcal{O}(n)$

❖ 挑战: 不依赖递归机制, 如何实现后序遍历? 效率如何?



观察



藤缠树

❖ 从根出发下行

尽可能沿**左**分支

实不得已，才沿**右**分支

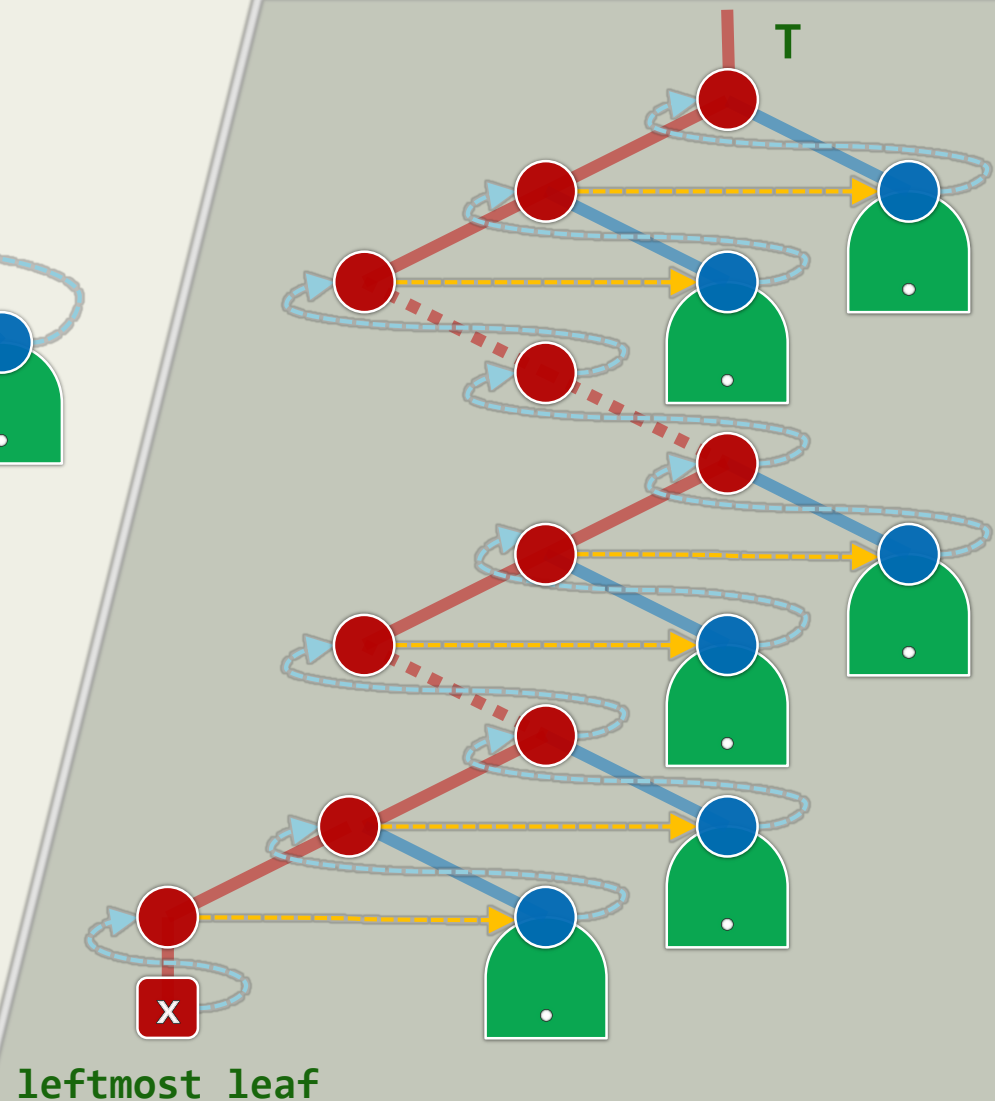
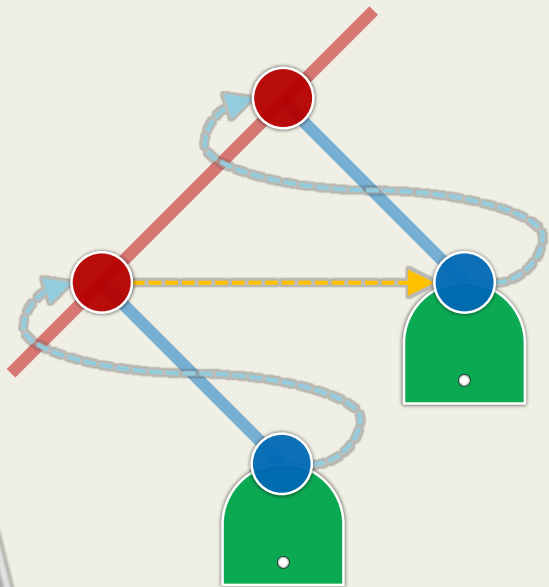
❖ 最后一个节点

必是叶子，而且

是按中序遍历次序**最靠左者**

也是递归版中visit()首次**执行**处

❖ 这片叶子，将首先接受访问...



序曲

❖ `template <typename T> static void gotoLeftmostLeaf( Stack <BinNodePosi<T>> & S ) {`

`while ( BinNodePosi<T> x = S.top() ) //自顶而下反复检查栈顶节点`

`if ( HasLChild( * x ) ) { //尽可能向左。在此之前`

`if ( HasRChild( * x ) ) //若有右孩子，则`

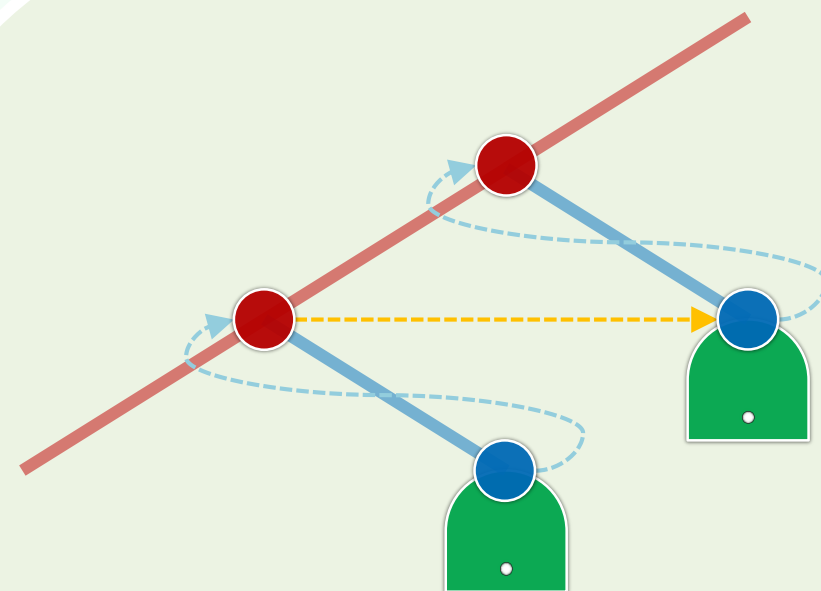
`S.push( x->rc ); //优先入栈`

`S.push( x->lc ); //然后转向左孩子`

`} else //实不得已`

`S.push( x->rc ); //才转向右孩子`

`S.pop(); //返回之前，弹出栈顶的空节点`



全曲

❖ template <typename T, typename VST>

```
void travPost_I( BinNodePosi<T> x, VST & visit ) {
```

```
    Stack < BinNodePosi<T> > S; //辅助栈
```

```
    if ( x ) S.push( x ); //根节点首先入栈
```

```
    while ( ! S.empty() ) { //x始终为当前节点
```

```
        if ( S.top() != x->parent ) //若栈顶非x之父（而为右兄）
```

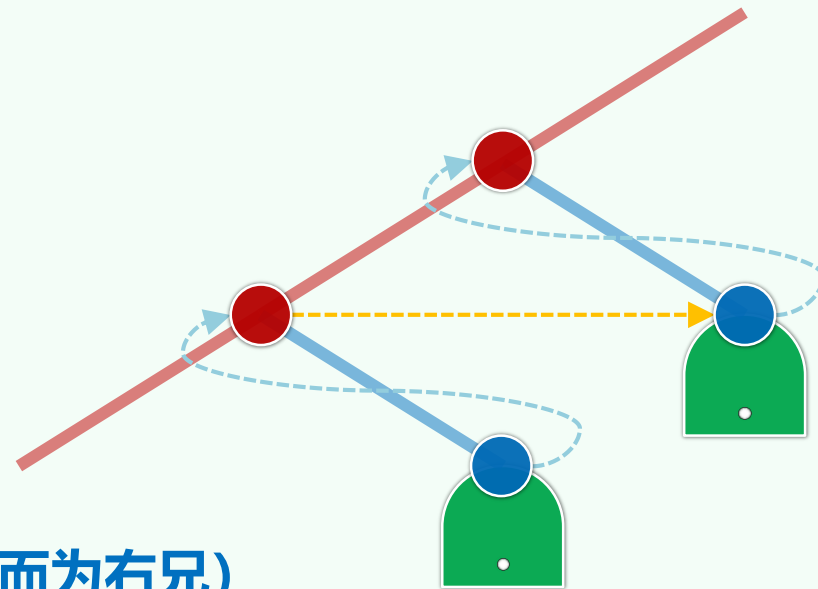
```
            gotoLeftmostLeaf( S ); //则在其右兄子树中找到最靠左的叶子（递归深入）
```

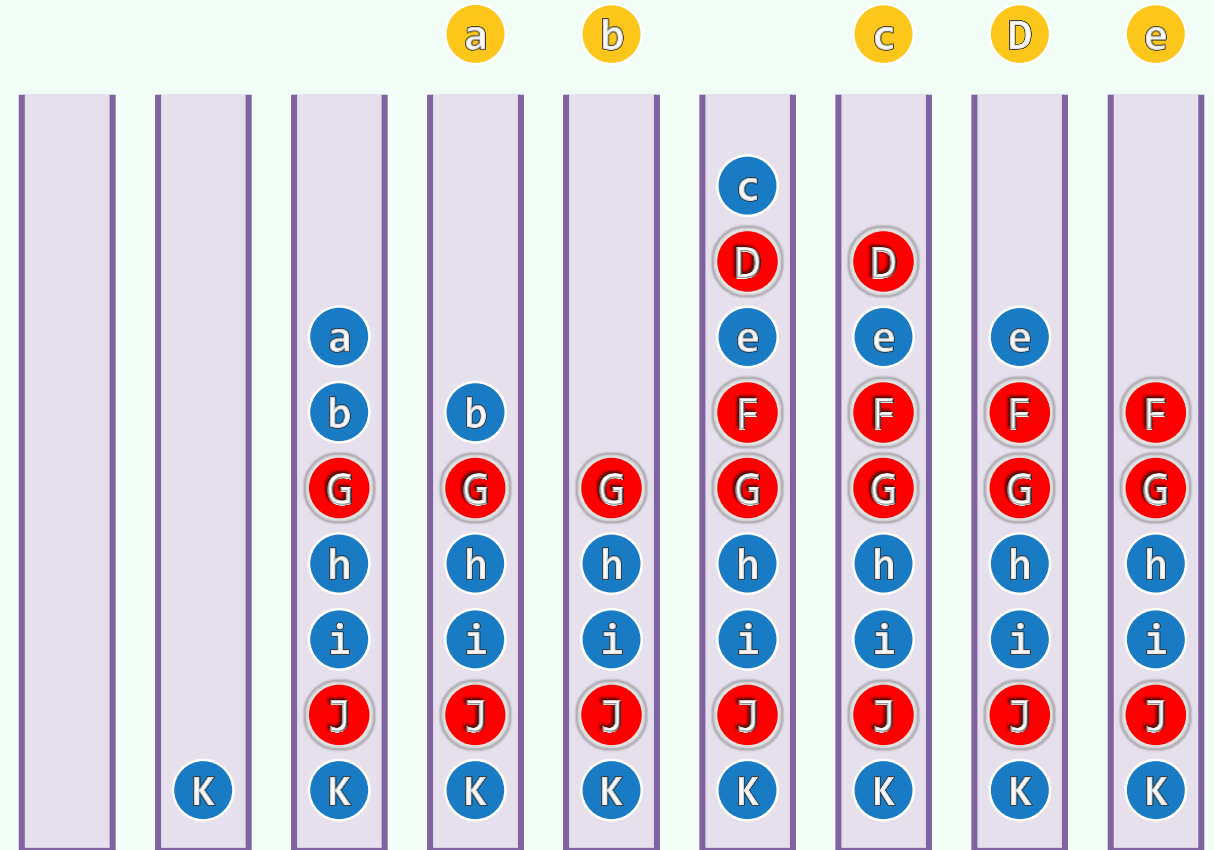
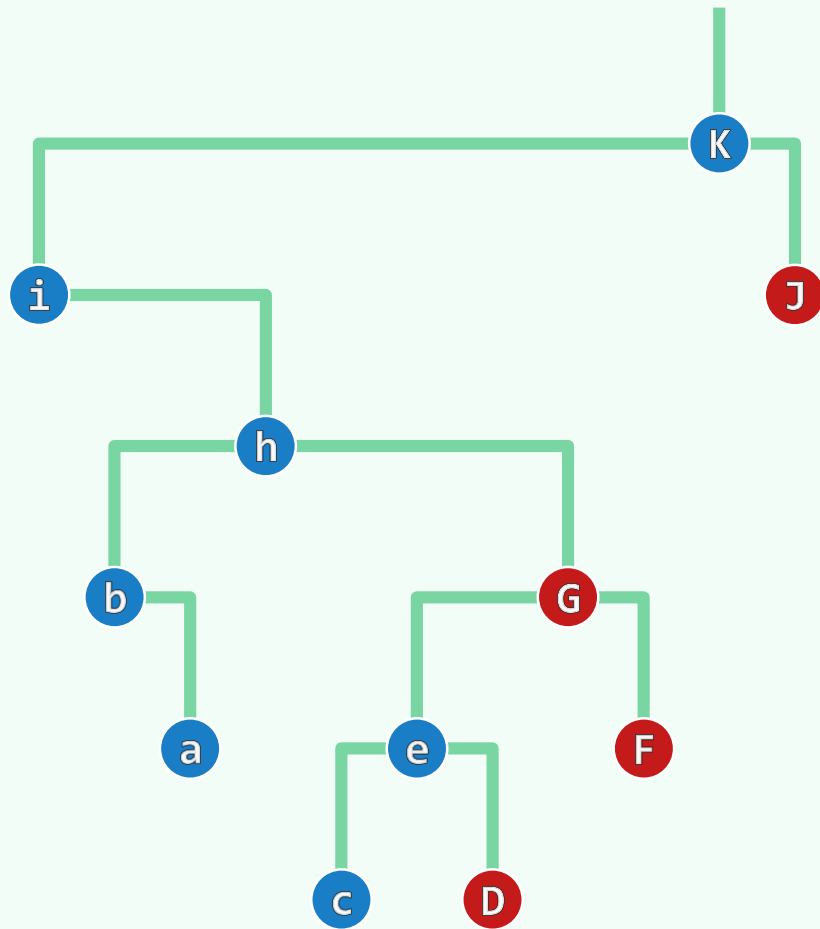
```
        x = S.pop(); //弹出栈顶（即前一节点之后继）以更新x
```

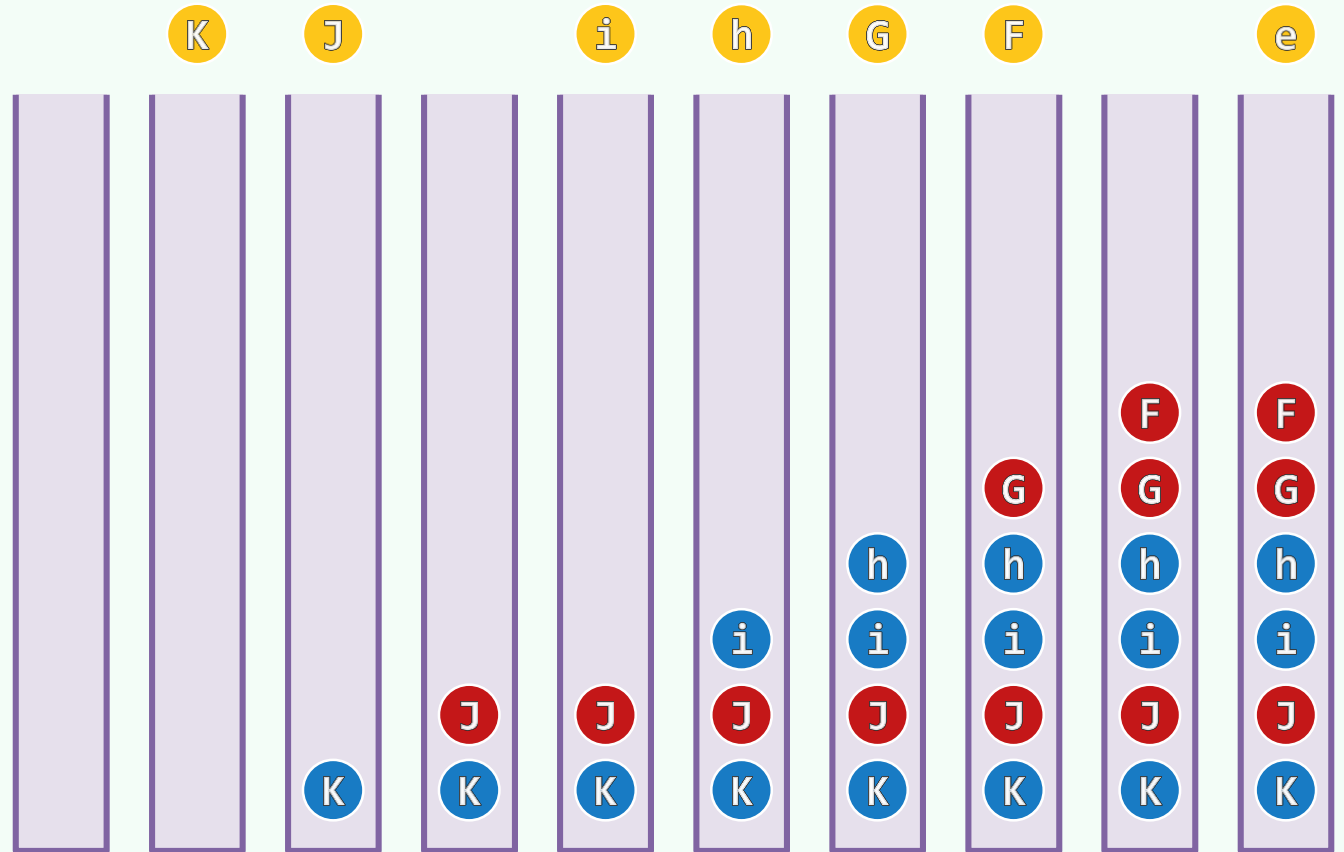
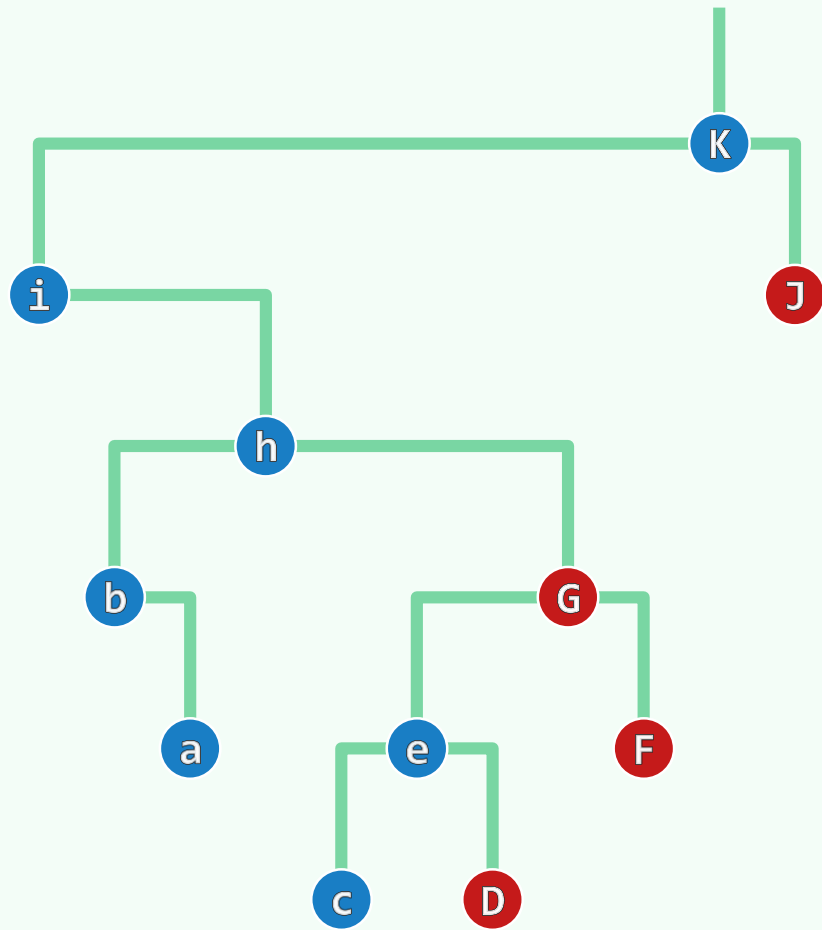
```
        visit( x->data ); //并随即访问之
```

```
    }
```

```
}
```







二叉树
后序遍历： 表达式

Expression

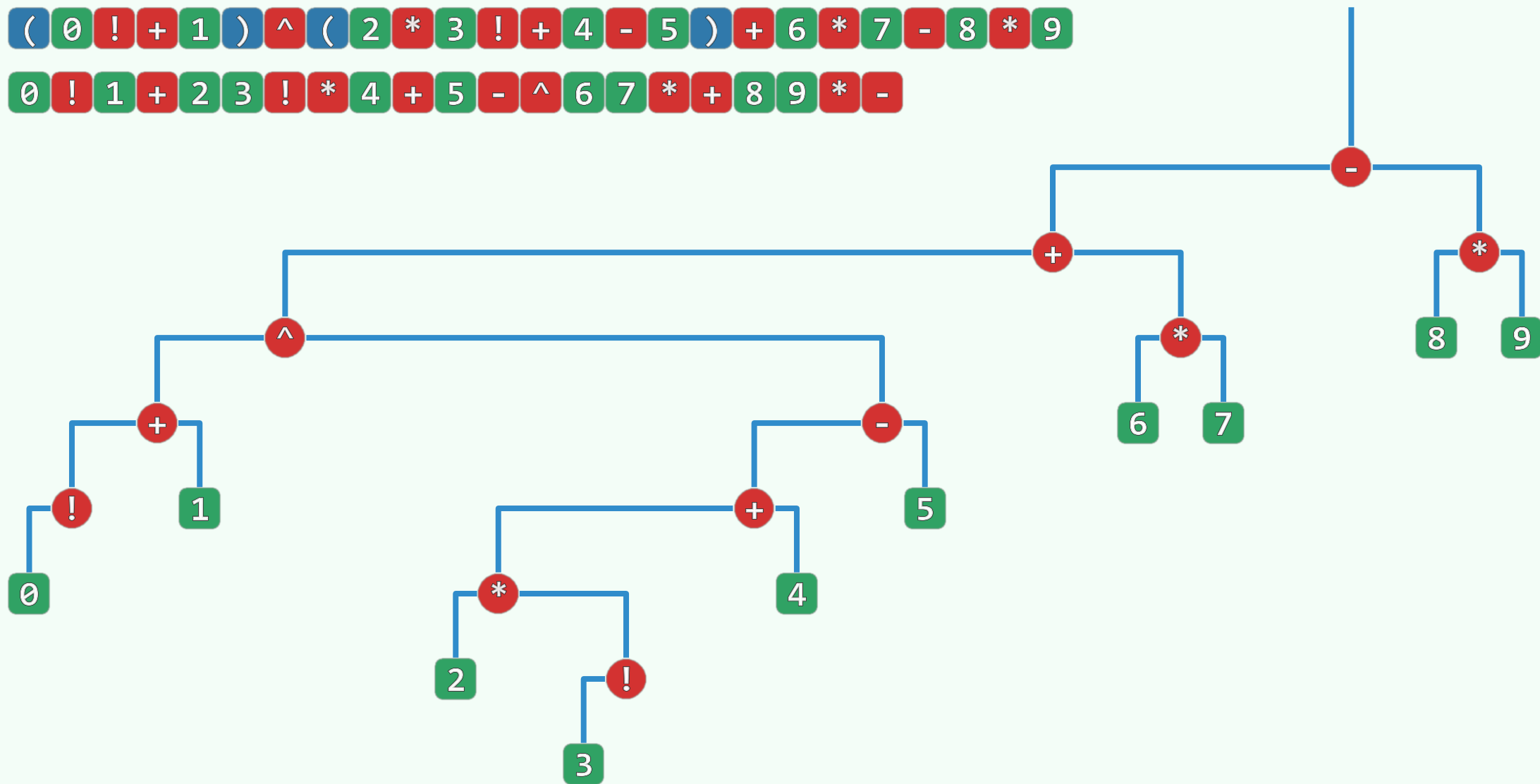
The diagram illustrates the step-by-step construction of a parse tree for the expression $(0!+1)^{(2*3!+4-5)+6*7-8*9}$. The root node is the full expression. It branches into $(0!+1)$, $^$, and $(2*3!+4-5)+6*7-8*9$. Further branching shows the derivation of the factorial and arithmetic sub-expressions, with leaf nodes being the tokens 0, 1, 2, 3, 4, 5, 6, 7, 8, 9, +, -, *, ^, and parentheses.

Expression Tree ~ Post-order ~ RPN

逆波兰表达式是后序遍历结果

(0 ! + 1) ^ (2 * 3 ! + 4 - 5) + 6 * 7 - 8 * 9

0 ! 1 + 2 3 ! * 4 + 5 - ^ 6 7 * + 8 9 * -



二叉树 层次遍历：算法与分析

算法实现

❖ template <typename T> template <typename VST>

```
void BinNode<T>::travLevel( VST & visit ) { //二叉树层次遍历
```

```
    Queue< BinNodePosi<T> > Q; //引入辅助队列
```

```
    Q.enqueue( this ); //根节点入队
```

```
    while ( ! Q.empty() ) { //在队列再次变空之前，反复迭代
```

```
        BinNodePosi<T> x = Q.dequeue(); //取出队首节点，并随即
```

```
        visit( x->data ); //访问之
```

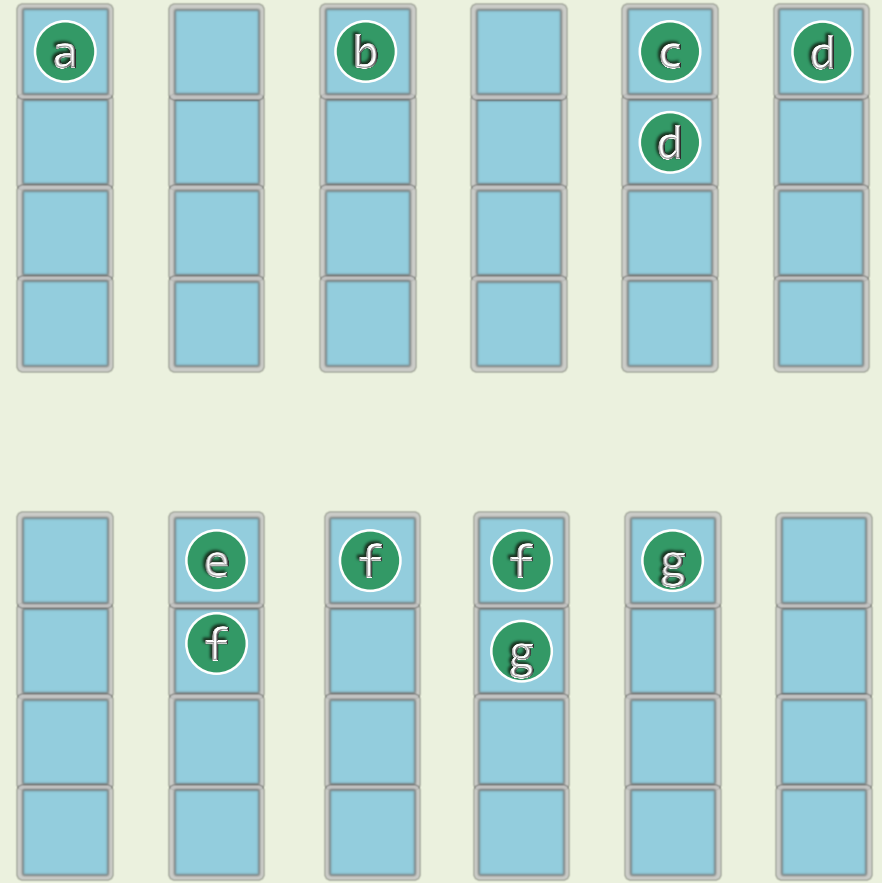
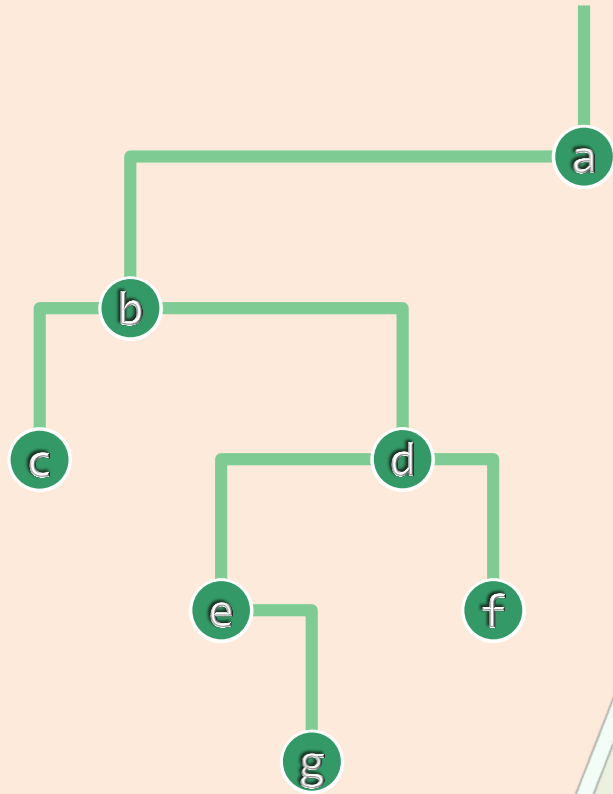
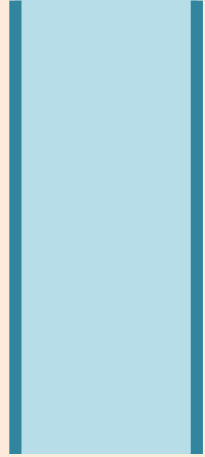
```
        if ( HasLChild( * x ) ) Q.enqueue( x->lc ); //左孩子入队
```

```
        if ( HasRChild( * x ) ) Q.enqueue( x->rc ); //右孩子入队
```

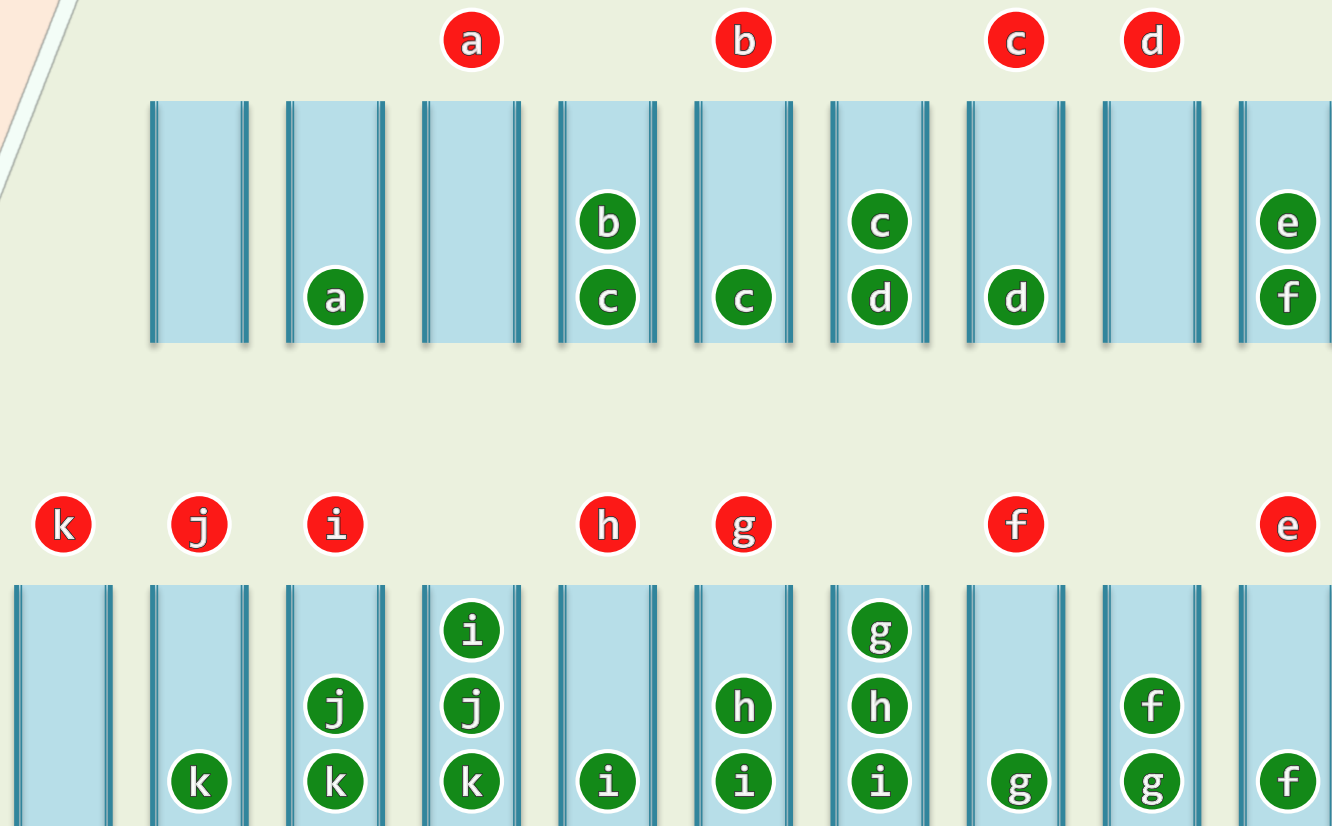
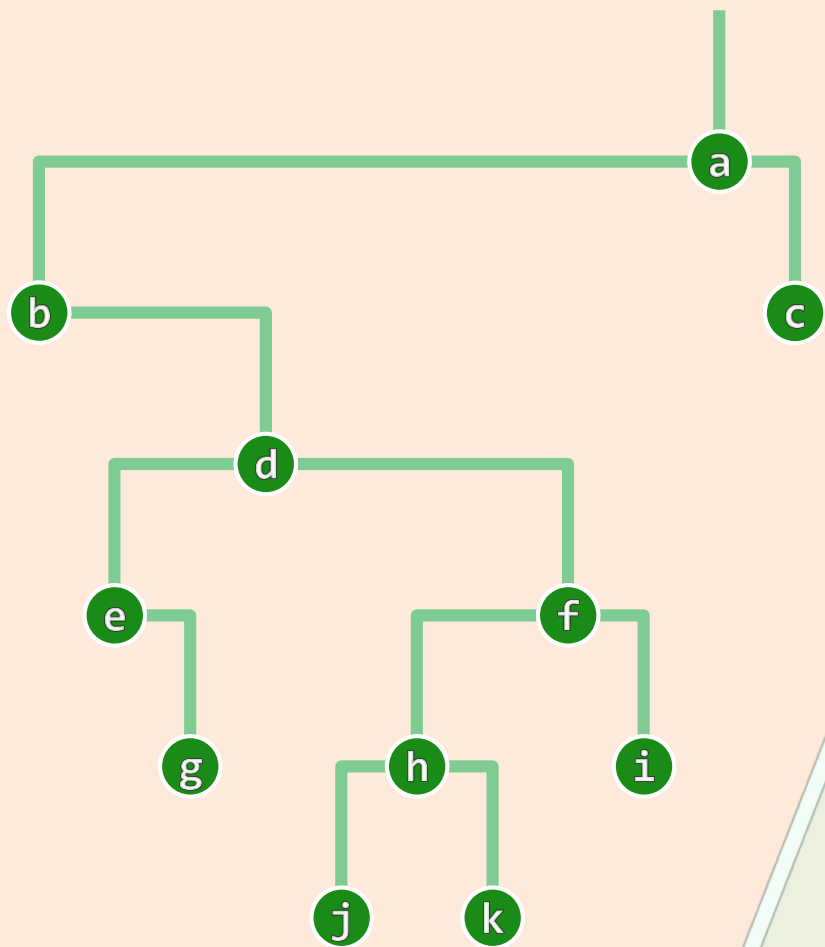
```
    }
```

```
}
```

实例

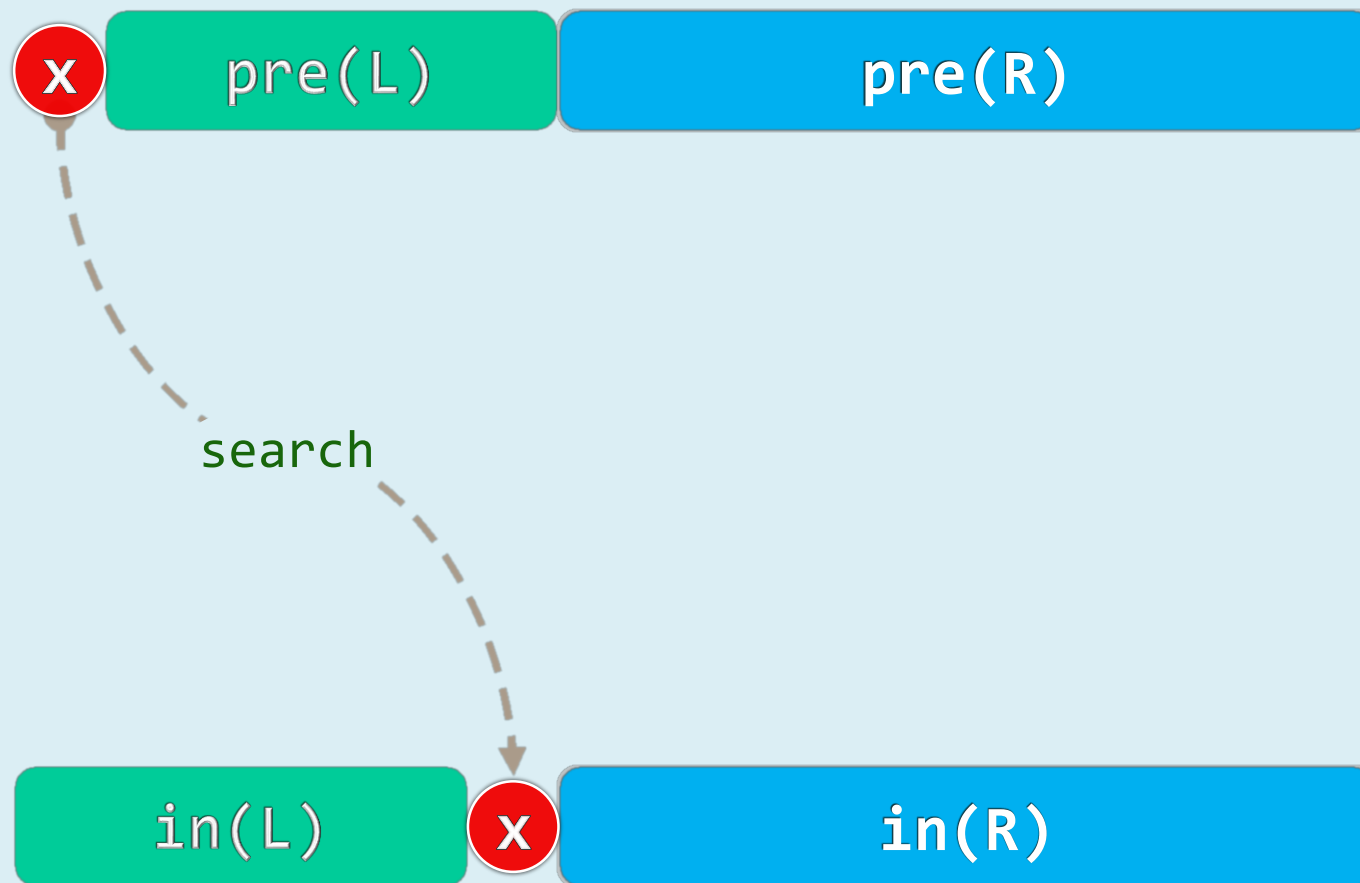
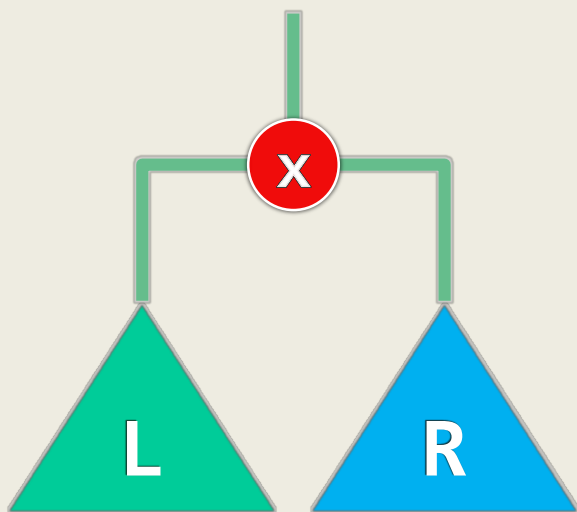


实例

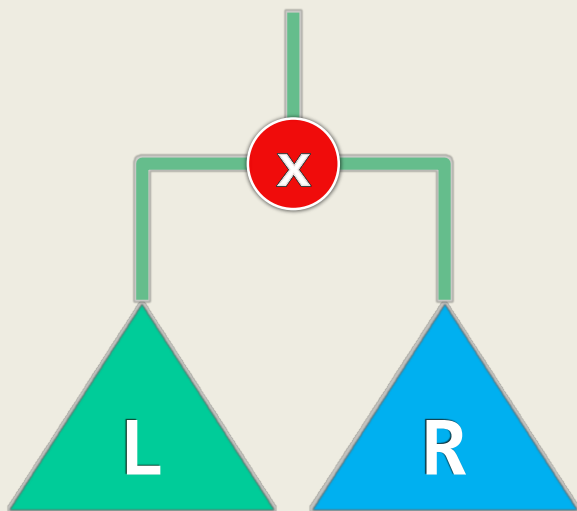


二叉树 重构

[先序 | 后序] + 中序



[先序 + 后序] ?



X

pre(L)

pre(R)

where to divide?

post(L)

post(R)

X

[先序 + 后序] !

